

DREW'S DRIVE

Offline AI-Based Vehicle-to-Vehicle Collision Avoidance and Driver Safety System

A Technical Research and Design Documentation Report

Submitted for MSME 6.0 Hackathon

Abstract

Road traffic accidents continue to be one of the leading causes of preventable deaths and injuries across the world. The situation is even more serious on highways, rural roads, hilly regions, and tunnels where mobile networks and internet connectivity are often unreliable or completely unavailable. Most modern connected-vehicle and driver-assistance systems depend on continuous cloud connectivity, making them less effective in the very places where timely warnings are needed the most.

Drew's Drive is a fully offline Vehicle-to-Vehicle (V2V) safety system designed to overcome this limitation. Built around a single ESP32-WROOM-32 microcontroller in each vehicle, the system enables direct communication between nearby vehicles without relying on the internet or any external infrastructure. It combines data from an MPU6050 motion sensor, a SIM28 GPS module, and an MQ-3 alcohol sensor to continuously monitor driving behaviour, detect possible alcohol impairment, and identify accident events using sensor fusion techniques.

This report explains the complete design and implementation of the platform, including its layered software architecture for sensor monitoring, driver behaviour analysis, risk assessment, offline hazard mapping, secure evidence logging, and authenticated peer-to-peer communication using ESP-NOW. It also describes the key algorithms used in the system, such as Time-to-Collision (TTC) prediction, confidence-based accident detection, GPS-loss dead reckoning, and pothole detection using dual-impulse analysis. These algorithms are implemented through carefully designed thresholds and state machines within the firmware.

The report also discusses the security features of the platform. Each vehicle generates a unique identity and authentication key from a shared fleet root key and its factory MAC address. Secure communication is achieved using AES-128-GCM encryption, HMAC-SHA-256 authentication, and a sliding-window replay protection mechanism to prevent unauthorized or repeated packet transmission.

As part of the development process, the firmware underwent a detailed verification and testing phase. This review identified and fixed eight critical issues, including a checksum calculation error that invalidated transmitted packets, a remotely exploitable buffer overflow vulnerability, and a watchdog configuration problem that caused the device to reboot repeatedly after start up. The report further presents the firmware's memory usage, FreeRTOS task distribution across both ESP32 cores, system validation results, current limitations, and possible future improvements.

Overall, Drew's Drive demonstrates that essential connected-vehicle safety features—including driver behaviour monitoring, alcohol impairment detection, accident detection through sensor fusion, collision warning, and tamper-resistant digital black-box logging—can be implemented on an affordable ESP32 microcontroller without relying on internet connectivity. This makes it a practical and cost-effective safety solution, especially for regions where communication infrastructure is limited or unavailable.

Keywords: Vehicle-to-Vehicle Communication (V2V), ESP-NOW, ESP32, Sensor Fusion, Collision Avoidance, Time-to-Collision, Driver Behaviour Analysis, Offline Navigation, Digital Black Box, Embedded Security, AES-GCM, FreeRTOS.

Introduction

1.1 Background

The past decade of automotive safety engineering has centered around two general strategies: on-board ADAS (advanced driver-assistance) systems relying on local cameras, radar, and inertial sensors for reasoning about the vehicle's immediate environment, and connected vehicle (V2X) technologies that facilitate state sharing between vehicles, infrastructure, and cloud services to enable reasoning beyond the sensor range of any single vehicle. While the latter is strictly more powerful where available (e.g. Drew's Drive focuses on a more modest, feasible question: How much of the safety benefits of connected vehicles can be achieved with a low cost microcontroller, some simple sensors, and a low-range P2P radio (i.e., without using the Internet or cell-phone based networks)?

This assumption breaks exactly where V2X is most desperately needed.

In most large portions of the developing world and a significant amount of arterial road in developed countries, hill corridors and tunnels are often entirely devoid of mobile signal. An automotive safety system that silently reverts to individual-vehicle ADAS at precisely those times it is most critical isn't a viable solution. Drew's Drive seeks to answer a narrower, more achievable problem: to determine how much of the benefit of connected vehicle safety can be achieved from just a low-cost microcontroller, a few basic sensors, and short-range P2P radio, with no dependence on cellular or the Internet.

1.2 Motivation

There are three primary motivators for this project.

Firstly, the ESP32 microcontroller offers a dual core processor, hardware AES/SHA acceleration, and an 802.11 radio capable of ESP-NOW (a connectionless, low-latency P2P protocol) on a single, low-cost module that is amenable to volume deployment.

This hardware provides more than enough computational and security resources for local sensor fusion and secure, authenticated communications, without the need for a dedicated external security chip. Secondly, many aspects of collision and impairment detection do not require cloud scale ML – simple threshold and state-machine based fusion of accelerometer, gyroscope, GPS, and even basic gas sensor readings can be sufficient to differentiate an accident/rollover event from severe braking, provided multiple confirming sensor samples are collected over a short time span, which this approach does. Thirdly, low cost retrofit matters: a system that can be installed in existing vehicles for the cost of a handful of components will be far more widely deployable, particularly in low-income markets, than a system requiring factory integration or an ongoing subscription service.

1.2 Problem Statement

Road safety statistics show that a few recurring, readily-detectable causal factors are responsible for a disproportionately high fraction of accidents: excess speed for conditions, aggressive maneuvering, driver impairment, delays in accident discovery, and lack of advance warning among vehicles nearing a common hazard. Current solutions for each of these problems tend to be disparate and have their own limitations: Dashcams provide evidence but do not prevent collisions; drunk driving interlocks confirm

impairment but do not notify other road users; cloud-connected telematic boxes can detect collisions but become inoperable without a data link. No widely-deployed, low-cost solution exists today that simultaneously

- (a) evaluates and scores driving behaviour,
- (b) identifies potential driver impairment with a non-invasive sensor,
- (c) confirms actual accidents via sensor fusion (rather than simply spike detection),
- (d) broadcasts its state and warnings to surrounding vehicles without reliance on external network infrastructure,
- (e) preserves an irrefutable log of events for accident reconstruction, all within a single, affordable embedded module

.

1.3. Goals

- Design and implement a vehicle to vehicle (V2V) offline safety platform in which only ESP-NOW is used as the sole means of vehicle-to-vehicle communication.
- Consistently monitor driving behaviour with inertia based sensors. Use the data to establish a restricted and comprehensible driver score.
- Use a stable gas semiconductor detector that verifies probable alcohol impairment. Multiple samples from the detector are used for verification, instead of only a single sample that could be corrupted or misleading.
- Verify probable vehicular accidents (collision, rollover, severe crash) with behavioural, inertial, and position information, using a bounded notification period and disregarding false alarms.
- Provide estimated position and direction, even during a GPS disruption, through inertial dead reckoning; the estimated confidence diminishes smoothly over time until it can be restored by GPS resynchronization.
- Warn surrounding vehicles about dangers, lane changing, impairment, and confirmed accidents using a secure and replay-protected communication.
- Maintain a offline, in-vehicle database with facility information and hazard locations. This will provide a vehicle driver with advanced warning of potentially dangerous zones, and directions to the nearest emergency facility or station.
- Create a protected digital log of all vehicle activity in the several moments preceding a confirmed accident or particularly rough ride, protected by cryptographic hash to prevent tampering, verified with each system startup.
- Through structured code reviews, static analysis, and targeted testing, guarantee the platform's code is free of critical defects, specifically those common to security-conscious embedded systems like memory corruption vulnerabilities, logical errors in protocol implementation, and service availability problems such as watchdog timer deactivation.

1.4 Project Scope

This document focuses on the design of firmware and the overall system architecture of a Drew's Drive vehicular unit, how the units interface with each other via ESP-NOW, and the offline data resources employed. -This current document deliberately excludes the vehicular integration process (e.g., car harness testing, EMC compliance, crashworthiness standards for the unit's enclosure) as well as any applications related to the road infrastructure or external devices. -The platform is intentionally designed to operate autonomously within the car, with plans for roadside recipient and vehicle fleet management tools being described solely as areas for future expansion. -The information presented pertains to the present firmware version, which secures telemetry through a shared symmetric fleet key-from which per-vehicle keys are derived-using AES-128-GCM for confidentiality and HMAC-SHA-256 for integrity; an optional ECDH-based commissioning protocol is outlined as a potential enhancement that replaces the original provisioning method without affecting the wire protocol.

1.5 Organization

- ❖ Chapter 1: provides an overview of relevant vehicle telematics and driver monitoring systems, placing the Drew's Drive system within this context.
- ❖ Chapter 2: lists functional and non-functional requirements.
- ❖ chapter 3: describes the system's overall architecture and its module call graph.
- ❖ Chapter 4: addresses the physical hardware components and vehicular wiring.
- ❖ Chapter 5: explains FreeRTOS task assignments.
- ❖ Chapter 6: goes through the implementation details for each firmware subsystem individually.
- ❖ Chapter 7: dives deep into the security architecture.
- ❖ Chapter 8 covers the core algorithms, such as those used for calculating time-to-collision, correlating various pieces of evidence to confirm accidents, using dead reckoning for navigation, and discriminating false pothole impacts from real ones.
- ❖ Chapter 9: details the management of persistent, offline data storage.
- ❖ Chapter 10: documents system testing, validation efforts, and identifies defects found and corrected during the review process.
- ❖ Chapters 11: through 15 present the system's results, benefits and limitations, future research directions, and a conclusion, respectively. - The appendices offer a quick-reference summary of information and sample code snippets.

2. Related Work and Comparative Study

2.1 Cloud-Connected Telematics and Fleet Safety Systems

Typical fleet management devices use hardwired sensors or OBD-II hardware and connect via cellular to transmit trip details, impact data, and vehicle location information to a central database. This enables broad historical analytics and a wide view of the fleet but requires a constant cellular signal for operation; any loss of connection means these features, along with vehicle-to-vehicle warning capabilities, are unavailable. These systems are inherently single vehicle-centric, since transmission through the cloud infrastructure introduces substantial lag, often in the seconds to minutes range, which is inadequate for collision avoidance purposes, which typically demand at least several seconds of warning time.

2.2 Dedicated Short-Range Communication (DSRC) and Cellular-V2X (C-V2X)

The established V2V communications standards, DSRC (IEEE 802.11p) and C-V2X, support ultra-low latency vehicle to vehicle communication and require minimal infrastructure, representing a standard solution for comprehensive, certified V2V safety messaging. However, DSRC necessitates specialized radio hardware that is largely absent in today's vehicles, and C-V2X requires access to cellular carrier spectrum and network participation. Both technologies remain largely inaccessible or prohibitively expensive for individual owners seeking to equip older or economy vehicles with an affordable after-market safety system.

2.3 Smartphone-Based Driver-Monitoring Applications

Consumer grade driver monitoring software often exploits a phone's accelerometer, gyroscope and GPS sensors to detect harsh driving and even crashes, triggering alerts or sending calls. Although this approach offers an economical solution with no need for additional hardware, it suffers from inherent disadvantages for safety applications. Firstly, phone connection reliability is often similar to that of in-car communication systems in areas with poor signal coverage, and the inconsistent placement of the device (in a pocket, cup holder, or mounted on a dash) significantly undermines the accuracy of critical threshold-based metrics, such as cornering forces or rollover angles. A vehicle-mounted, stable inertial sensor and a dedicated ESP-NOW interface for peer-to-peer messaging, as incorporated into Drew's Drive, overcome both these limitations.

2.4 Alcohol-Impairment Detection

The established technology for detecting alcohol impairment is a breathalyzer with an ignition interlock. However, these devices require driver interaction and may only catch impaired driving at specific times of day, failing to detect alcohol-induced impairment that might develop later or that isn't triggered when the driver is initially screened. The non-invasive semiconductor gas sensor (like the MQ-3 used in this project) offers continuous, in-vehicle measurement but is not legally certified and sensitive to airborne alcohol from various sources.

Drew's Drive therefore uses the gas sensor as an indication of general impairment risk rather than an evidentiary tool.

Measurements must be stabilized over multiple consecutive readings before escalating risk, and the documentation clearly states that the results are an aid to the driver, not a medical or legal finding.

2.5 Positioning the Present Work

- Zero dependency on connectivity: every critical safety feature – driver behaviour analysis, hazard warnings, crash detection, offline mapping and route planning – operates in isolation without external communication.

- The system incorporates multi-signal collision detection, using diverse inputs rather than relying on a single criterion to avoid erroneous triggering by impacts like hitting potholes or kerbs. Secure and tamper-proof V2V communication using existing ESP32 hardware, eliminating the need for dedicated V2X radios.
- A tamper-resistant, cryptographical - signed digital recorder stores vehicle data, verified after each power cycle, ensuring data integrity and historical playback in the event of an accident.
- An offline, crowd-sourced map and navigation system providing access to facility routing, even without an internet connection.

Table 1 summarises this positioning against the three related-work categories discussed above.

Table 1. Comparison of Drew's Drive against related vehicle-safety approaches.

Capability	Cloud telematics	DSRC / C-V2X	Smartphone apps	Drew's Drive
Works with zero connectivity	No	Yes (infra-optional)	No	Yes
Peer-to-peer warning latency	Seconds–minutes	< 100 ms	Seconds–minutes	~100 ms (10 Hz broadcast)
Dedicated hardware required	Modem + module	Certified V2X radio	None (uses phone)	Commodity ESP32 + sensors
Fixed, known sensor orientation	Usually yes	N/A	No	Yes
Tamper-evident evidence log	Cloud-side only	Varies	Rare	On-device, hash-verified
Approx. incremental unit cost	Medium–high + data plan	High	None (uses owned phone)	Low, one-time

3.System Requirements

3.1 Functional Requirements

- FR-1: The system shall sample triaxial acceleration and angular rate at 50 Hz and derive roll and pitch via a complementary filter.
- FR-2: The system shall acquire GPS fixes via NMEA sentences at up to 20 Hz sentence rate and expose position, speed, heading, and UTC-derived epoch time.

- FR-3: The system shall sample the MQ-3 alcohol sensor and convert its analogue output into an estimated blood-alcohol percentage after a mandatory warm-up period.
- FR-4: The system shall compute a bounded (0-100) driving score from harsh acceleration, harsh braking, sharp turning, aggressive steering, swerving, and unsafe lane-change events.
- FR-5: The system shall classify alcohol risk into Warming-Up, Normal, Possible, and Confirmed-High states using stability-gated multi-sample confirmation.
- FR-6: The system shall confirm accident events (possible collision, confirmed collision, rollover, severe impact) using fused inertial, positional, and behavioural evidence with a bounded confirmation countdown, and shall continue IMU-only detection when GPS is unavailable.
- FR-7: The system shall estimate position, heading, speed, and distance travelled through GPS outages using inertial dead reckoning, and shall track and expose a decaying confidence value for estimated positions.
- FR-8: The system shall detect potholes from vertical acceleration impulses distinct from routine road vibration and persist detected locations to an offline hazard map.
- FR-9: The system shall broadcast authenticated telemetry, lane-warning, pothole-warning, hazard-warning, emergency-alert, and impaired-driver-warning messages to nearby vehicles over ESP-NOW, prioritised by severity.
- FR-10: The system shall receive and authenticate the same message types from nearby vehicles, reject replayed or forged packets, and track nearby vehicle state for time-to-collision evaluation.
- FR-11: The system shall compute time-to-collision against every tracked nearby vehicle and escalate the local alert level when a critical or warning threshold is crossed.
- FR-12: The system shall maintain an offline hazard map (school zones, curves, speed breakers, railway crossings, accident-prone zones, potholes) and warn the driver on proximity, factoring current speed into the warning distance.
- FR-13: The system shall support offline routing requests to the nearest hospital, police station, fire station, or fuel station from a locally provisioned dataset.
- FR-14: The system shall continuously compute a vehicle health score from GPS, IMU, gas-sensor, V2V, storage, and battery status.
- FR-15: The system shall maintain a rolling 60-second, 5 Hz sample buffer and freeze it into a hashed, tamper-evident evidence file upon a confirmed accident or a severe driving event.
- FR-16: The system shall verify every stored evidence file's integrity at boot and report a verified or integrity-failed state.

- FR-17: The system shall present current speed, driver score, risk level, alcohol state, GPS status, V2V status, active hazards, navigation status, and evidence integrity on a local TFT dashboard, and shall present a distinct full-screen emergency view when the alert level is Emergency.

3.2 Non-Functional Requirements

- NFR-1 (Availability): The system shall recover from any single sensor fault without a full reset and shall reflect the fault in the vehicle health score rather than halting other subsystems.
- NFR-2 (Real-time bounds): Inertial sampling, accident evaluation, and dead reckoning shall run at 50 Hz; V2V broadcast and time-to-collision evaluation at 10 Hz; the dashboard shall refresh at 4 Hz; all four FreeRTOS tasks shall service the hardware watchdog on every loop iteration.
- NFR-3 (Confidentiality and integrity): All inter-vehicle telemetry shall be encrypted and authenticated such that a passive radio observer cannot recover plaintext and an active attacker cannot forge or replay a packet accepted by a receiver.
- NFR-4 (Resource footprint): The firmware shall fit within the default 1,310,720-byte application partition and shall not exceed available static DRAM on the ESP32-WROOM-32 (520 KB total SRAM, of which 320 KB is readily available to the application in the default partition layout).
- NFR-5 (Tamper evidence): Evidence files shall be individually hashed such that any post-hoc modification is detectable at the next boot without requiring an external reference copy.
- NFR-6 (Fleet consistency): Every vehicle in a fleet shall share a single pre-provisioned root key and protocol version; a version or key mismatch shall cause packets to be silently rejected rather than mis-parsed.
- NFR-7 (Field maintainability): Hazard and facility datasets shall be editable in the field as plain CSV files without recompiling firmware, and firmware updates shall not require the offline datasets to be re-provisioned.

3.3 Hardware Requirements Summary

Table 2. Minimum hardware bill of materials per vehicle unit.

Component	Purpose	Interface
ESP32-WROOM-32 (mhetesp32devkit)	Dual-core application processor, Wi-Fi/ESP-NOW radio, AES/SHA acceleration	—
MPU6050	6-axis accelerometer + gyroscope for behaviour and accident sensing	I2C, 400 kHz
SIM28 GPS receiver	Position, speed, heading, UTC time	UART, 9600 baud NMEA
MQ-3 gas sensor	Ambient-cabin alcohol vapour presence/trend estimation	Analogue (12-bit ADC)
2.4-inch SPI TFT (ILI9341, 240×320)	Local driver dashboard	SPI (VSPI)

Component	Purpose	Interface
Active buzzer	Audible warning / emergency tone	Digital GPIO
Warning and critical LEDs	Visual escalation indicators	Digital GPIO
Resistive battery divider	Supply-voltage health monitoring	Analogue (12-bit ADC)

3.4 Software Requirements Summary

- PlatformIO build system with the Arduino-ESP32 framework.
- Espressif32 platform providing core 3.x Arduino APIs (the community-maintained pioarduino distribution is used in place of the official espressif32 platform definition where core 3.x support is required).
- LittleFS for on-device storage of the hazard database, facility database, and black-box evidence files.
- mbedTLS (bundled with the ESP-IDF/Arduino-ESP32 core) for AES-GCM, HMAC-SHA-256, and SHA-256 primitives.
- TinyGPS++ for NMEA sentence parsing.
- TFT_eSPI for the ILI9341 dashboard driver.
- FreeRTOS (bundled with ESP-IDF) for task scheduling, mutexes, and inter-task queues.

4. System Architecture and Design

4.1 Architectural Overview

Drew's Drive is organised as a single-process, multi-task firmware image in which every subsystem is a small, independently testable C++ class owning its own mutex-protected state, exposing a snapshot accessor that copies its current state out under lock, and an update method invoked by exactly one of four FreeRTOS tasks. This design keeps cross-task data races scoped to a single, auditable lock per object rather than a single global lock, while still allowing every task to read a fully consistent snapshot of any subsystem's state without blocking that subsystem's own update path for long.

At the top of the object graph, `main.cpp` owns one instance of each subsystem and wires them together purely through function calls that pass snapshots by value — SensorHub produces a `SensorData` snapshot; Driver Behaviour Analyzer consumes it and produces a `Driver Behaviour Snapshot`; Safety Coordinator consumes both and internally owns `Position Estimator`, `Alcohol Risk Engine`, `Accident Detector`, and `Driver Risk Engine`, producing a single `Safety Snapshot`; `Hazard Map`, `Offline Navigation`, `Collision Engine`, `Vehicle Health Monitor`, `Digital Black Box`, `V2VManager`, and `Dashboard` then consume whatever combination of these snapshots they need. No subsystem reaches into another subsystem's internals — the only channel between them is the snapshot struct returned by that subsystem's own accessor.

4.2 High-Level Data Flow

Figure 1 shows the module dependency graph as extracted directly from the firmware's include structure, rooted at `src/main.cpp`. Sensor acquisition (`sensors/SensorHub.cpp`) feeds three consumers: driver-behaviour analysis, pothole detection, and the safety coordinator. The safety coordinator, in turn, is the hub from which position estimation, alcohol risk, accident detection, and driver risk all flow downstream into V2V broadcast (`comm/V2VManager.cpp`, which itself delegates cryptography to `security/SecurityManager.cpp`), the offline hazard map, offline navigation, the digital black box, and the TFT dashboard.

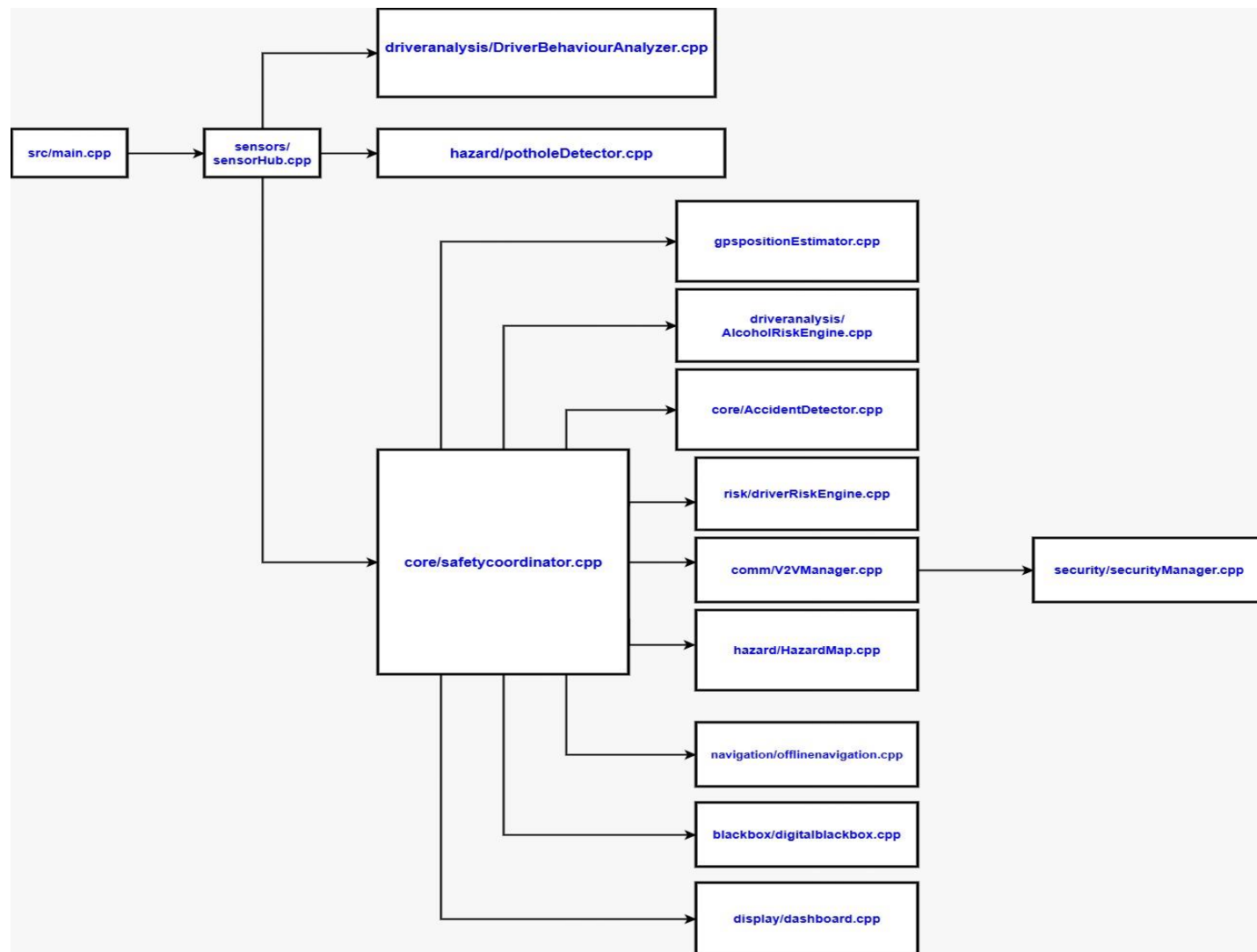


Figure 1. Firmware module dependency graph, generated from the project's include hierarchy rooted at `src/main.cpp`.

4.3 Layered View

- Sensing layer — `sensors/SensorHub.cpp`: raw acquisition and calibration for IMU, GPS, MQ-3, and battery.
- Analytics layer — `driveranalysis/DriverBehaviorAnalyzer.cpp`, `driveranalysis/AlcoholRiskEngine.cpp`, `hazard/PotholeDetector.cpp`: turns raw samples into scored events.
- Fusion / safety layer — `core/SafetyCoordinator.cpp` orchestrating `gps/PositionEstimator.cpp`, `core/AccidentDetector.cpp`, `risk/DriverRiskEngine.cpp`: turns scored events plus position into a single safety verdict.
- Situational-awareness layer — `hazard/HazardMap.cpp`, `navigation/OfflineNavigation.cpp`, `risk/CollisionEngine.cpp`, `tracking/VehicleTracker.cpp`: combines local and peer-sourced information into hazard and collision warnings.
- Communication and security layer — `comm/V2VManager.cpp`, `security/SecurityManager.cpp`: authenticated broadcast/receive over ESP-NOW.
- Persistence layer — `blackbox/DigitalBlackBox.cpp`: evidence capture, hashing, and verification on LittleFS.
- Presentation and actuation layer — `display/Dashboard.cpp`, `drivers/AlertManager.cpp`: TFT rendering and buzzer/LED escalation.
- Health layer — `health/VehicleHealthMonitor.cpp`: a cross-cutting view of every other subsystem's operational status.

4.4 Design Principles Observed in the Codebase

4.4.1 Snapshot isolation

Every stateful class exposes a `snapshot()` method that takes its internal FreeRTOS semaphore, copies its state struct by value, releases the semaphore, and returns the copy. Consumers therefore never hold a lock across their own processing, which keeps lock hold times short and bounded and eliminates lock-ordering deadlocks between subsystems, since no subsystem ever needs to hold two locks at once.

4.4.2 Bounded, fixed-size state

There is no dynamic heap allocation in the hot paths: the nearby-vehicle table, the replay-protection table, the hazard-record table, the facility table, and the black-box ring and frozen buffers are all fixed-size arrays sized by compile-time constants in `config/Config.h`. This is a deliberate embedded-systems discipline that removes heap fragmentation as a failure mode over long uptimes and makes worst-case memory usage statically knowable.

4.4.3 Explicit event consumption

Several subsystems expose a `consume*()` method (`consumeNewEvents`, `consumeConfirmation`, `consumeHighConfirmation`, `consumeAccidentConfirmation`, `consumeAlcoholConfirmation`) rather than a plain boolean flag read. Each of these atomically reads and clears a pending-edge flag under the subsystem's own lock, guaranteeing that a one-shot event such as "accident just confirmed" is observed exactly once by the network task even though multiple tasks poll the subsystem's snapshot every cycle.

4.4.4 Shared capacity constant

`config/Config.h` documents that `MAX_TRACKED_VEHICLES` sizes both the nearby-vehicle table in `VehicleTracker` and the per-vehicle replay-protection table in `SecurityManager` from the same constant, specifically to prevent the two structures from silently drifting apart as the fleet size assumption changes — a small but telling example of the codebase encoding an invariant directly in a shared constant rather than in a comment alone.

5. Hardware Design

5.1 Controller: ESP32-WROOM-32 (mhetesp32devkit)

This platform specifically targets the `mhetesp32devkit` board, which features a dual-core Xtensa LX6 processor capable of running the Arduino-ESP32 framework at 240 MHz. It offers 520KB of on-chip SRAM and integrates 802.11 b/g/n Wi-Fi connectivity. Two aspects of the ESP32-WROOM-32 are crucial to the system's design: firstly, the dual-core architecture allows separate tasks like sensor fusion to operate on one core without impacting networking or storage tasks, ensuring no task starves the watchdog timer. Secondly, the hardware-accelerated AES and SHA functionality (accessible via `mbedtls`) make encrypting outbound packets with AES-128-GCM and authenticating them with HMAC-SHA-256 computationally feasible at the 10 Hz broadcast rate without significantly taxing the CPU.

5.2 Sensors

5.2.1 MPU6050 inertial measurement unit

The MPU6050 is a triaxial accelerometer and angular rate sensor that communicates over I2C at 400 kHz. Upon startup, the firmware calibrates the accelerometer and gyroscope by taking 100 stationary readings to determine bias. This bias is then subtracted from all subsequent measurements.

The firmware employs a complementary filter to fuse the data from both sensors, producing estimates for roll and pitch that remain stable even when subjected to vibrations (which would destabilize pure accelerometer data) and free from long-term drift (unlike pure gyroscope data).

Raw acceleration data is collected at 50 Hz.

5.2.2 SIM28 GPS receiver

The SIM28 is a UART NMEA GPS/GNSS receiver, operating at the standard 9600 baud rate. TinyGPS++ is used by the firmware to parse the NMEA sentences. This parser is polled at up to 50 Hz within the loop.

A validated GPS fix is obtained only when the location, date, and time fields are all reported as valid by the parser during the same update.

The health status of the GPS receiver is also monitored independently. A `gpsReceiverHealthy` flag is maintained, indicating whether any NMEA sentences have been received within a 5-second timeout; this distinguishes "receiver present but not yet connected" from "receiver has stopped communicating with the microcontroller."

5.2.3 MQ-3 alcohol sensor

The MQ-3 is a resistive semiconductor gas sensor that detects ethanol vapor. Its response is characterized in the datasheet, allowing the firmware to estimate blood-alcohol-percentage based on the sensor's analog output. At boot, the firmware takes 64 readings to establish the clean-air resistance of the sensor (R_0).

Subsequent readings measure the ratio of the current sensor resistance to R_0 and use a log-linear approximation of the datasheet's characteristic curve to generate a BAC estimate.

Due to the inherent noise and contamination susceptibility of the MQ-3, the sensor's output is used for a presence/trend classifier (see Section 7.3) rather than for direct pass/fail decisions based on individual readings.

5.2.4 Battery-voltage sense

The battery voltage is applied to an ADC-only GPIO via a resistive divider, configured to have a nominal 2:1 ratio. The firmware never directly connects the battery to an ADC pin, making the divider mandatory. Low-voltage conditions are also penalized by the vehicle-health score.

5.3 Display and Human-Machine Interface

A 240×320 SPI ILI9341 TFT panel, driven through `TFT_eSPI`, presents the local dashboard. The panel is configured over VSPI with a 27 MHz SPI clock and glyph fonts loaded for the numeric and label rendering used throughout the dashboard. An active buzzer and two discrete LEDs (warning, amber; critical, red) provide non-visual escalation so that warnings remain perceivable even when the driver is not looking at the panel.

5.4 Wiring Reference

Table 3. Pin assignment for the `mhetesp32devkit` build target, consolidated from `platformio.ini` and `config/Config.h`.

Signal	ESP32 GPIO	Notes
MPU6050 SDA	21	I2C data, 400 kHz
MPU6050 SCL	22	I2C clock, 400 kHz
SIM28 GPS TX → ESP32 RX	16	9600 baud NMEA, UART2
SIM28 GPS RX ← ESP32 TX	17	9600 baud NMEA, UART2
TFT MOSI	23	VSPI
TFT MISO	19	VSPI
TFT SCLK	18	VSPI, 27 MHz
TFT CS	5	ILI9341 chip select
TFT DC	26	Data/command
TFT RST	33	Reset
TFT Backlight	32	Digital drive, active high
MQ-3 analogue output	34	Input-only ADC pin
Battery divider output	35	Input-only ADC pin, 2:1 divider expected
Active buzzer	25	Logic-level active buzzer
Warning LED	14	Use a suitable current-limiting resistor
Critical LED	27	Use a suitable current-limiting resistor

The firmware's mounting-orientation assumption — X axis forward, Y axis left, Z axis up — is load-bearing for every threshold documented in Chapter 9, and the README explicitly requires this orientation to be preserved consistently across every vehicle in a fleet so that a single set of behavioural and accident thresholds remains valid fleet-wide.

6. FreeRTOS Task Architecture

6.1 Task Partitioning

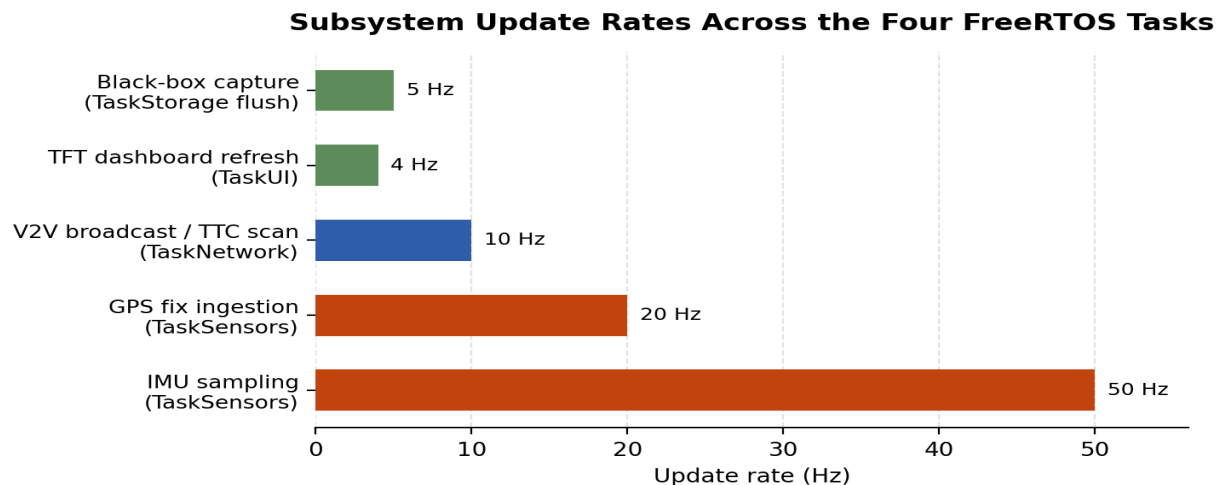
The firmware runs exactly four pinned FreeRTOS tasks, each registered with the hardware watchdog (`esp_task_wdt`) and resetting it on every loop iteration. Task placement across the ESP32's two cores is chosen so that time-critical sensing and the radio/network path never compete for the same core.

Table 4. FreeRTOS task allocation, priority, and stack size, as created in `setup()`.

Task	Core	Priority	Stack (bytes)	Cadence
Task Sensors	1	2	4096	IMU 50 Hz, GPS ≤ 50 Hz poll, MQ-3 5 Hz, loop delay 5 ms
Task Network	0	2	8192	Broadcast/TTC/black-box cycle 10 Hz, prune 1 Hz, loop delay 10 ms
TaskUI	1	1	6144	Dashboard refresh 4 Hz, alert pattern update every tick, loop delay 25 ms
Task Storage	0	1	4096	Black-box flush poll, loop delay 100 ms

Task Sensors and TaskUI share core 1 because the UI is a low-priority, comparatively infrequent consumer that must never block the 50 Hz inertial sampling loop on which accident and behaviour detection ultimately depend; Task Network and Task Storage share core 0 because encryption, ESP-NOW transmission, and LittleFS writes are all naturally bursty I/O-adjacent operations that are better isolated from the sensing core entirely.

Figure 2. Update rate of each subsystem, grouped by the FreeRTOS task that drives it.



6.2 Task Sensors

On each iteration, Task Sensors independently rate-limits three sub-updates using local `millis()` timers: IMU sampling every 20 ms (50 Hz), GPS polling every 50 ms, and MQ-3 sampling every 200 ms (5 Hz). After each sub-update it immediately calls `Safety Coordinator::update Fast()`, which internally re-evaluates alcohol risk on every call but only re-runs position estimation and accident detection when a new IMU sample has actually landed (guarded by comparing `Sensor Data::imuLastUpdateMs` against the previous value), avoiding redundant fusion work on GPS- or MQ-3-only ticks.

6.3 Task Network

Task Network is the busiest task in the system. On its 10 Hz cycle it: drains any received V2V messages and folds pothole/hazard reports from peers into the local hazard map; runs `SafetyCoordinator::updatePeriodic()` to refresh the driver-risk score; updates the hazard-proximity and navigation-distance calculations against the latest position; scans every tracked nearby vehicle through `CollisionEngine::predictTTC()`; computes the overall Alert Level from accident status, collision state, harsh-braking, lane-change, risk level, and alcohol state; updates the vehicle-health score; feeds the digital black box's rolling sample buffer and, when triggered, its event lock; and finally enqueues zero or more outbound V2V messages (pothole warning, lane warning, emergency alert, impaired-driver warning, and routine telemetry) in priority order. A separate one-second cycle prunes stale entries from the nearby-vehicle table. Every cycle ends with a call to `comm.processTx()`, which drains the outbound priority queue at a maximum cadence of one packet every 20 ms.

6.4 Task UI

Task UI polls the USB-serial console for offline-navigation commands (``nav hospital``, ``nav police``, ``nav fire``, ``nav fuel``, ``nav clear``), drives `AlertManager::update()` on every tick so LED/buzzer patterns stay phase-accurate regardless of the 25 ms task delay, and refreshes the TFT dashboard at 4 Hz by gathering fresh snapshots from every subsystem and calling `Dashboard::render()`.

6.5 Task Storage

Task Storage's sole responsibility is to poll `DigitalBlackBox::flushPending()` at 10 Hz effective (100 ms task delay) so that a frozen evidence buffer produced by Task Network after a confirmed accident is written to LittleFS — with the atomic write-temp-then-rename pattern described in Chapter 7 — outside of the network task's own time budget.

6.6 Watchdog Discipline

Every one of the four tasks calls `esp_task_wdt_add(NULL)` once at task entry and `esp_task_wdt_reset()` at the end of every loop iteration, with a global timeout of 5000 ms configured in `setup()` via `esp_task_wdt_init(timing::WDT_TIMEOUT_MS, true)`. Because the parameter for panic-on-timeout is passed as true, a task that stalls for longer than the timeout — for example, a blocking I2C call that never returns — triggers a full panic and reboot rather than allowing one wedged task to silently starve the others while appearing to run. Correct registration of every task with the watchdog subsystem was, as documented in Chapter 11, one of the defects found and corrected during the review of this codebase.

7. Software Module Reference

This chapter documents every firmware subsystem in the order data flows through it: sensing, behavioural and impairment analytics, positional and accident fusion, situational awareness, communication and

security, persistence, and presentation. Each section states the class's responsibility, its key state, and the exact thresholds or formulas it applies, drawn directly from the current implementation.

7.1 Sensor Hub ([sensors/SensorHub.cpp](#))

The Sensor Hub is the single source of truth for all raw peripherals. `Begin()` initialises I2C at 400 kHz with a 50ms bus timeout (to prevent a stuck I2C bus from hanging the sensing task indefinitely), opens UART2 for the GPS at 9600 baud, wakes the MPU6050 from sleep and performs the two startup calibration passes outlined in Chapter 5. `UpdateImu()` converts raw 16-bit accelerometer and gyroscope registers to physical units (m/s and degrees/sec), subtracts the calibrated bias, calculates a gravity-compensated `verticalLinearAccel` value exclusively for pothole detection, updates the complementary filtered roll/pitch estimate and sets `imuHealthy` to false only after 3 consecutive I2C read failures or 1 full second of no sample – a deliberate debouncing approach that tolerates occasional lost I2C packets without flickering the health status. `UpdateMq3()` converts the analogue reading to a sensor-resistance ratio and then to an estimate BAC percentage, and simultaneously estimates battery percentage by dividing the supply rail.

7.2 Driver Behaviour Analyzer ([driver analysis/DriverBehaviorAnalyzer.cpp](#))

This subsystem translates IMU readings to 6 distinct, independently-cool downed event types – harsh acceleration, harsh braking, sharp turning, aggressive steering, swerving, and unsafe lane change – only when the vehicle speed exceeds a minimum threshold of 12 km/h, preventing stationary vibrations or parking maneuvers from triggering a driving event. Swerving is detected using a state machine based on yaw-rate sign changes; it takes three direction reversals within 1400 ms for a swerving event to trigger, thus distinguishing genuine weaving from a single hard turn. Each detected event incurs a fixed penalty on a 0-100 driving score (harsh accel 4, harsh brake 5, sharp turn 3, aggressive steer 6, swerving 8, unsafe lane change 10) and enters a 2500ms cooldown during which the same event type is prevented from triggering again, although the display remains illuminated for another 1800ms after the cooldown begins for a brief moment to avoid transient event flickering on the dashboard. The score regenerates at 1 point/sec whenever no event is occurring, and the current driving style is categorized as Calm, Normal, Aggressive or High Risk solely based on the active score and whether any event is ongoing.

7.3 Alcohol Risk Engine ([driver analysis/AlcoholRiskEngine.cpp](#))

The Alcohol Risk Engine specifically treats the MQ-3 as a non-certified presence sensor. Until 60 seconds past the last boot and with an electrical self-check completed, only the “Warming-Up” state is possible. If and when the sensor has warmed up, its reading is considered “stable” if it has changed by no more than 0.012 percentage points since the previous reading; only stable readings advance the possible-risk counter (against a 0.03% threshold) or the high-risk counter (against a 0.08% threshold). The “Possible” state is achieved if either three stable possible-level readings occur, or a single stable possible-level reading coincides with a current driving score below 60, on the rationale that erratic driving often accompanies intoxication. The “Confirmed-High” state requires five stable high-level readings, and is the only state to issue a one-shot flag

that the network task consumes to broadcast an authenticated warning message to proximate vehicles

7.4 Position Estimator (gps/PositionEstimator.cpp)

The Position Estimator is the primary provider of “where-is-this-vehicle” information to all other firmware modules, be it a real-time GPS fix or a dead-reckoned estimate. When `data.gpsValid` is true, the estimator snaps to the reported GPS location, speed and heading and resets confidence to 100%. When GPS is lost, the estimator advances state: heading is integrated forward using the gyroscope’s yaw rate and elapsed time; speed is advanced using the forward accelerometer axis and elapsed time, then is decaying forward by a constant factor of 0.998 per tick to counter unbounded drift in uncompensated accelerometers; and position is integrated forward using the current heading and estimated distance in a flat-Earth equirectangular projection. The status is set to Weak while the last real fix was within the last 5 seconds and to Lost beyond that, and position confidence degrades at 1 and 2 points per tick while Weak and Lost, respectively, to provide downstream components-particularly the Accident Detector, which scales its output confidence by 25% in this mode-a constantly-degrading level of trustworthiness.

7.5 Accident Detector (core/AccidentDetector.cpp)

This module represents the highest-severity fusion in the firmware and is elaborated upon in detail in Section 9.2. It continuously analyses 5 distinct physical inputs per IMU cycle – impact severity, severe impact severity, angular velocity, rate of deceleration, and roll/pitch angle – to derive a combined confidence value, requiring both multiple indicators and a minimum number of supporting samples over a ten-second interval to transition from “Monitoring” through “Confirming” to “Confirmed.” Upon being “Confirmed,” the state is sticky until externally reset, and a one-shot confirmation flag initiates both the emergency V2V broadcast and the black-box recording.

7.6 Driver Risk Engine (risk/DriverRiskEngine.cpp)

The Driver Risk Engine combines the driving behaviour score, the alcohol-risk state, the harsh-braking event flag and the accident state into a single 0-100 risk score and a Safe, Moderate, Dangerous, or Critical rating. Its components include 55% of the inverse driving behaviour score, supplemented by a fixed penalty for an active unsafe lane change (12), swerving (10), or aggressive steering (8); 45% of a high alcohol risk score or 20% of a possible risk score; 8 points for a harsh-braking event in the current cycle; and a binary trip for an accident. When a Confirmed accident is active, the risk score is clamped to 100 regardless of other contributions. This calculation results in a human-interpretable score composed of readily inspectable terms.

7.7 Collision Engine (risk/CollisionEngine.cpp)

This module estimates the time-to-collision with each nearby detected vehicle using a simple constant-velocity model; this algorithm is elaborated upon in Section 9.1. As a stateless static utility, it stores no persistent data, and is called once per detected vehicle during each pass through the network task.

7.8 Hazard Map (hazard/HazardMap.cpp)

Hazard Map owns a fixed 48-record table of hazard entries (accident-prone zones, sharp curves, speed breakers, railway crossings, school zones, and locally or remotely detected potholes), persisted to LittleFS as a length-prefixed, CRC-32-checked binary store (`/hazards.db`) written through an atomic temp-file-then-rename sequence identical in structure to the black-box evidence writer. On first boot, or whenever the binary store fails its magic/version/CRC check, the map is seeded by importing `/hazards.csv` from the LittleFS data partition. Adding a hazard that duplicates an existing record within a 12-metre radius merges into the existing entry (keeping the higher severity and refreshing the timestamp) rather than creating a duplicate; when the table is full, the oldest record by timestamp is evicted to make room. Proximity warnings scale their lookahead distance with current speed — a minimum 45-metre floor plus three seconds of travel at the current speed — so that a warning is issued with roughly constant time-to-hazard rather than constant distance-to-hazard regardless of how fast the vehicle is moving.

7.9 Pothole Detector (hazard/PotholeDetector.cpp)

Pothole Detector is discussed algorithmically in Section 9.4. It runs a continuously updated exponential baseline of vertical acceleration and looks specifically for the two-impulse rebound signature of a real wheel-drop event, gated to GPS-valid conditions above 8 km/h, with a seven-second cooldown between detections to avoid re-reporting the same pothole repeatedly as the vehicle continues over a rough patch of road.

7.10 Offline Navigation (navigation/OfflineNavigation.cpp)

Offline Navigation loads a fixed 32-entry facility table (hospital, police, fire, fuel) from `/places.csv` at boot and, on a `request()` call — issued either from the serial console or, in a full deployment, a physical button — searches the table for the nearest facility of the requested type on every subsequent position update, reporting distance and name, or an explicit "no local record" result if the field-provisioned dataset does not cover the current area. The design intentionally avoids inventing or interpolating facility locations; a location is only ever offered if it was explicitly provisioned for that deployment.

7.11 Vehicle Health Monitor (health/VehicleHealthMonitor.cpp)

Vehicle Health Monitor is a pure aggregator: on every network-task cycle it recomputes a 0–100 health score from fixed penalties — IMU unhealthy (–40, and forces the Critical level regardless of the numeric score), MQ-3 unhealthy (–15), V2V not ready (–25), storage not ready (–20), GPS not valid (–10), and low battery voltage between 0.2 V and 3.35 V, i.e. present but under-voltage rather than disconnected (–10) — and classifies the result into Healthy, Degraded, Fault, or Critical bands. This score is itself broadcast in every outbound telemetry packet, giving nearby vehicles visibility into a peer's operational status, not just its position.

7.12 Digital Black Box (black box/DigitalBlackBox.cpp)

The black box maintains a 300-sample (60 second at 5 Hz) ring buffer of compact, packed samples — position, speed, event flags, acceleration, angular rate, BAC, driving score, risk score, alert level, GPS status, alcohol level, and accident status — captured on every network-task cycle by `record()`, subject to the 200 ms sampling interval and the buffer being unlocked. `lockAfterEvent()`, called when the safety coordinator reports a fresh accident confirmation or a severe driving event, atomically copies the entire ring into a `frozen` array, builds a `CrashEvidence` summary record, and sets a pending-flush flag; once locked, the ring buffer stops accepting new samples until the lock is externally cleared, guaranteeing the frozen evidence is never overwritten by post-event samples. `flushPending()`, invoked by the dedicated storage task, computes a SHA-256 hash over the file header (excluding the hash field itself) and every sample, writes the result to a temporary file, and only then removes any prior file at that slot and renames the temporary file into place — an atomic-rename pattern that guarantees a reader never observes a partially written evidence file even across a power loss mid-write. At boot, `scanCrashFiles()` re-verifies every one of the three rotating slots by recomputing the same hash and comparing it against the stored value, counting verified and failed files separately and selecting the next write slot as the one immediately after the most recently verified file, so that new evidence never overwrites the newest still-verified record before older ones

7.13 Security Manager (security/SecurityManager.cpp)

Security Manager is treated in full in Chapter 8. In brief, it derives a network-wide AES key and a per-vehicle authentication key and identity from a single pre-shared fleet root key and each device's factory Wi-Fi MAC address, encrypts and HMAC-authenticates every outbound packet, and verifies MAC-to-identity binding, HMAC, AES-GCM tag, packet age, and per-sender replay window on every inbound packet.

7.14 V2VManager (comm/V2VManager.cpp)

V2VManager owns the ESP-NOW lifecycle: it fixes the Wi-Fi channel to channel 6, registers a single broadcast peer (`FF:FF:FF:FF:FF:FF`) with link-layer encryption explicitly disabled — because, as documented directly in the source, application-layer AES-GCM already protects the broadcast payload, making a second layer of ESP-NOW link encryption redundant — and registers send/receive callbacks. Outbound messages are queued in a small fixed-capacity priority queue (`enqueue()`); when the queue is full, a new message only displaces the current lowest-priority entry if its own priority is strictly higher, and `processTx()` always transmits the single highest-priority queued message roughly every 20 ms, so Emergency-priority packets are never starved behind a backlog of routine Information-priority telemetry. Inbound packets are handed to `SecurityManager::decryptAndVerify()`; only packets that pass every check are folded into the `VehicleTracker` and placed on a receive queue drained by the network task.

7.15 VehicleTracker (tracking/VehicleTracker.cpp)

VehicleTracker is a small fixed-capacity table (sized by the shared `MAX_TRACKED_VEHICLES` constant) mapping each authenticated peer's derived 16-byte vehicle ID to its most recently reported position,

speed, and heading, with a last-seen timestamp. `pruneStale()`, run once per second, drops any entry not refreshed within the configured stale window, so a vehicle that has moved out of radio range or powered off disappears from local time-to-collision evaluation within a bounded time rather than lingering indefinitely on stale coordinates

7.16 Alert Manager (`drivers/AlertManager.cpp`)

Alert Manager converts the single computed `AlertLevel` into physical LED and buzzer behaviour using only non-blocking, `millis()`-timed patterns so that the UI task's own loop cadence is never held up waiting on a `tone()` call. Safe and Warning levels are silent at the LED/buzzer level by design — Warning is dashboard-only, so that routine, low-severity guidance does not train the driver to associate every buzz with something serious. Danger produces a 500 ms-period LED flash together with a short, spaced 1800 Hz reminder tone (100 ms on, 800 ms period). Emergency produces a solid critical LED and a continuous 2000 Hz tone. Every level transition resets the internal pattern timer so a new level always starts its pattern from a clean phase rather than an arbitrary offset inherited from the previous level.

7.17 Dashboard (`display/Dashboard.cpp`)

Dashboard renders two distinct layouts depending on alert level. At any level below Emergency, it draws a header bar, a speed panel, a combined driver-score/risk panel, a GPS/navigation/V2V status strip, alcohol and accident status lines, a health/black-box summary line, a hazard/navigation panel, and a colour-coded reason strip along the bottom explaining the current alert in one phrase, prioritising accident confirmation, then confirmed impairment, then unsafe lane change, then any active hazard, then a generic "monitoring" state. At Emergency level, the entire screen is replaced with a full-red emergency layout showing crash type, confidence, combined risk/behaviour score, position status, evidence-integrity state, and an explicit confirmation that the emergency V2V broadcast is active, so a first responder or bystander glancing at the panel after a confirmed collision sees the single most important fact — that help has already been signalled to nearby vehicles — without needing to interpret the routine dashboard.

7.18 Safety Coordinator (`core/SafetyCoordinator.cpp`)

Safety Coordinator is the orchestration point that gives the rest of the system a single 'Safety Snapshot' (position, alcohol, accident, and risk state together) instead of four separately-timed subsystems to poll individually. `updateFast()`, called from the sensing task, always refreshes alcohol risk on every sensor update but only re-runs position estimation and accident detection when a genuinely new IMU sample has arrived, using the raw `imuLastUpdateMs` timestamp as the freshness signal rather than a separate dirty flag. `updatePeriodic()`, called from the network task, refreshes the driver-risk score, which is deliberately evaluated at the lower 10 Hz network cadence rather than the 50 Hz sensing cadence, since risk scoring depends on behavioural and alcohol state that itself only changes at a much lower rate.

7.19 Orchestration in main.cpp

main.cpp is intentionally thin: it owns one instance of every subsystem, performs boot-time initialisation (NVS flash init with erase-and-retry on a corrupted partition, watchdog init, LittleFS mount, and every subsystem's own `begin()`), creates the four pinned tasks described in Chapter 6, and then deletes the Arduino `loop()` task entirely (`vTask Delete(nullptr)` in `loop()`) since all real work happens in the explicitly created FreeRTOS tasks rather than the default Arduino loop. A small set of free functions in an anonymous namespace — `make Telemetry()`, `process Navigation Console()`, and `handle Received Messages()` — factor out payload construction, the serial navigation console, and inbound-message handling so that `Task Network`'s own body stays focused on sequencing rather than low-level detail.

8. Security Architecture

8.1 Threat Model

ESP-NOW broadcasts on the 2.4 GHz band with no physical access control: any device within radio range can passively receive every packet a vehicle transmits, and any device with knowledge of the wire format can attempt to inject forged or replayed packets. Because the payload includes position, speed, driving-risk, and even impaired-driver-warning data, both confidentiality and authenticity matter — a passive eavesdropper should not be able to reconstruct a vehicle's trip from captured broadcasts, and an active attacker should not be able to forge an Emergency-priority packet that pulls a nearby vehicle's dashboard into the full-screen emergency state, nor replay a previously valid packet to fabricate a stale hazard warning. SecurityManager (security/SecurityManager.cpp) is designed against exactly this threat model, using a symmetric, pre-shared-key architecture appropriate to hardware with no interactive pairing UI and no trusted third party reachable offline.

8.2 Key Hierarchy and Identity Derivation

Every vehicle in a fleet is provisioned before deployment with an identical 32-byte fleet root key (`security::FLEET_ROOT_KEY` in `config/Config.h`). No other secret is provisioned per vehicle; instead, three fleet-wide-context-tagged HMAC-SHA-256 derivations produce everything else a vehicle needs, using its own factory Wi-Fi MAC address as the only per-device input:

- A single, fleet-wide 128-bit AES key: HMAC-SHA-256(root key, "DrewsDrive/AES128/v5"), truncated to 16 bytes. This key is identical on every vehicle, which is what allows any receiver in the fleet to decrypt any sender's broadcast without a per-pair key exchange.
- A per-vehicle 256-bit authentication key: HMAC-SHA-256(root key, "DrewsDrive/Auth/v5" || MAC), used only for that vehicle's outbound HMAC and derivable by any receiver from the sender's MAC address for verification.

- A per-vehicle 128-bit identity: the first 16 bytes of HMAC-SHA-256(root key, "DrewsDrive/VehicleId/v5" || MAC), used as the vehicle's protocol-level ID in place of its raw MAC address.

This construction means a receiver never needs to have previously seen a sender to authenticate it: given the sender's MAC address (available from the ESP-NOW receive callback) and the shared root key, the receiver independently re-derives both the expected vehicle ID and the expected authentication key, and only accepts a packet whose claimed identity and HMAC match those independently derived values. There is no key exchange handshake and no per-pair state to establish before two vehicles can authenticate each other's broadcasts.

8.3 Wire Format

Every packet is a fixed-layout header (`WirePacket`), followed by the AES-GCM ciphertext of a fixed-size `TelemetryPayload`, followed by a 32-byte HMAC-SHA-256 trailer. Table 3 lists the header fields.

Table 5. *WirePacket header layout (security/SecurityManager.h).*

Field	Size	Purpose
magic	1 byte	Fixed marker (0xDD) rejecting non-protocol traffic early
protocol	1 byte	Must equal the fleet's compiled <code>PROTOCOL_VER</code> (currently 6)
type	1 byte	MessageType — Telemetry, LaneWarning, PotholeWarning, HazardWarning, EmergencyAlert, ImpairedDriverWarning
priority	1 byte	MessagePriority — Information, Warning, Critical, Emergency
vehicleId	16 bytes	Sender's derived identity
timestamp	4 bytes	Sender's GPS epoch seconds at send time (0 if GPS time unavailable)
sequence	4 bytes	Monotonic, NVS-persisted per-vehicle counter
nonce	12 bytes	vehicleId[0:8] big-endian sequence — deterministic and unique per (vehicle, sequence) pair
encLen	2 bytes	Declared ciphertext length; must equal <code>sizeof(TelemetryPayload)</code> exactly
tag	16 bytes	AES-GCM authentication tag

The `TelemetryPayload` itself carries position, speed, heading, raw acceleration and angular rate, epoch time, an event-flag bitmask, risk and driving scores, alcohol state, battery percentage, health score, hazard type/severity, crash type/confidence, and location confidence/status — everything a receiving dashboard or collision engine needs, in a single encrypted structure.

8.4 Send Path: Encrypt-then-Authenticate

`encryptAndSign()` populates the header, assigns the next sequence number (reserving a fresh block of sequence numbers in NVS whenever the live counter approaches its previously reserved ceiling, discussed in Section 8.6), derives the deterministic nonce, and calls `mbedtls\_gcm\_crypt\_and\_tag()` to encrypt the payload and produce the authentication tag using the fleet-wide AES key. Only after encryption succeeds does it compute an HMAC-SHA-256 over the entire header-plus-ciphertext byte range using the sender's own per-vehicle authentication key, appending the result as the final 32 bytes of the outbound buffer. This is a textbook encrypt-then-MAC construction, chosen over decrypt-then-verify ordering specifically because it lets a receiver reject a forged or corrupted packet purely from the HMAC check before ever invoking the more expensive AES-GCM decryption path.

```
// security/SecurityManager.cpp – encryptAndSign() (excerpt)
packet->sequence = txCounter++;
memcpy(packet->nonce, vehicleId, 8); // first 8 bytes: sender identity
packet->nonce[8] = uint8_t(packet->sequence >> 24); // last 4 bytes: big-endian sequence
packet->nonce[9] = uint8_t(packet->sequence >> 16);
packet->nonce[10] = uint8_t(packet->sequence >> 8);
packet->nonce[11] = uint8_t(packet->sequence);

mbedtls_gcm_crypt_and_tag(&gcm, MBEDTLS_GCM_ENCRYPT, sizeof(TelemetryPayload),
                        packet->nonce, sizeof(packet->nonce),
                        outBuf, offsetof(WirePacket, tag), // AAD: header up to tag
                        (const uint8_t*)&payload, ciphertextOut, sizeof(packet->tag),
                        packet->tag);

hmacSha256(vehicleAuthKey, sizeof(vehicleAuthKey), outBuf,
            sizeof(WirePacket) + sizeof(TelemetryPayload), hmacOut); // HMAC over header +
ciphertext
```

8.5 Receive Path: Verify-then-Decrypt

`decryptAndVerify()` performs seven checks, each of which must pass before the packet is accepted, in an order chosen to reject cheap-to-detect forgeries before spending cycles on cryptographic decryption:

- **Structural validity:** magic byte, protocol version, message type and priority enum ranges, and that the declared `encLen` exactly equals `sizeof(TelemetryPayload)` and that the total received length matches header + encLen + HMAC exactly — this single check is what closes the buffer-overflow class of defect discussed in Chapter 11, since a crafted, oversized `encLen` is rejected before any copy or decrypt is attempted.
- **MAC-to-identity binding:** the receiver re-derives the expected vehicle ID from the sender's link-layer MAC address (supplied out-of-band by the ESP-NOW callback, not by the packet payload) and rejects the packet outright if it does not constant-time-equal the vehicle ID claimed inside the packet — preventing a device from claiming another vehicle's identity while transmitting from its own MAC.

- HMAC verification: the receiver re-derives the sender's authentication key from that same verified MAC address and recomputes the HMAC over the received header-plus-ciphertext, comparing it to the trailer in constant time.
- Timestamp bound: if both sides have valid GPS epoch time, the packet is rejected if its age exceeds a 120-second bounded clock-skew window; if either side lacks GPS time, this check is skipped and replay protection alone carries the freshness guarantee.
- AES-GCM authenticated decryption: ``mbedtls_gcm_auth_decrypt()`` decrypts the payload and independently verifies the authentication tag, providing a second, cryptographically distinct integrity check beneath the HMAC.
- Replay-window check: the decrypted packet's sequence number is tested against a 256-message sliding-window replay filter maintained per sender vehicle ID (Section 8.7).
- Only if every prior check has passed are the decrypted payload, message type, priority, and vehicle ID returned to the caller.

8.6 Sequence-Counter Persistence

Because the GCM nonce is deterministic (identity || sequence) rather than random, nonce reuse — which would catastrophically break AES-GCM's confidentiality and integrity guarantees — must never occur for a given vehicle identity, including across a power loss. ``reserveCounterBlock()`` addresses this by writing a counter **ceiling** (the current counter plus a 1000-message reserve block) to NVS immediately whenever the live counter approaches its previously reserved ceiling, and by reading that ceiling back at boot as the starting counter value rather than resuming from a possibly stale last-committed value. The practical effect is that at most 1000 sequence numbers are ever "wasted" by an unclean shutdown, in exchange for guaranteeing the counter never repeats a value already used before a crash — an appropriate and common flash-wear-aware trade-off for a counter that must be crash-safe without committing to NVS on every single packet.

8.7 Replay Protection

Replay state is tracked independently per sender vehicle ID in a fixed-size table sized by the shared ``MAX_TRACKED_VEHICLES`` constant (Section 4.4.4) — a design correction discussed in Chapter 11, since an earlier revision of this logic tracked a single shared window across every sender, which meant the first vehicle heard from in a session would advance the window far enough that every other vehicle's legitimately lower sequence numbers were then rejected as replays. Each per-vehicle entry maintains a base sequence number and a 256-bit sliding bitmap; a received sequence number at or below the base is rejected outright, a number more than 256 ahead of the base slides the window forward and clears the bitmap, and any number already marked in the bitmap is rejected as a duplicate. A ``sequence == 0`` is

always rejected, since the counter is initialised to begin at 1 and a zero value can only arise from an uninitialised or corrupted field.

8.8 Link-Layer Encryption Decision

The ESP-NOW peer is deliberately registered with ``encrypt = false``. This is a considered choice rather than an oversight: ESP-NOW's own link-layer encryption operates per-peer and does not fit a broadcast topology well, and since every payload is already AES-128-GCM encrypted and HMAC-authenticated at the application layer, a second link-layer encryption pass would add CPU and latency cost without a corresponding increase in the actual security guarantee delivered to the application.

8.9 Forward-Looking Provisioning Path

The pre-shared root-key model is an appropriate choice for the prototype specifically because broadcast ESP-NOW offers no trusted interactive pairing channel on the hardware currently used. The design leaves an explicit path to strengthen provisioning without touching the wire protocol: an authenticated ECDH commissioning flow, run once over a physically-trusted setup channel at fleet enrolment time, could replace the manual root-key flashing step while leaving every field described in Section 8.3, and every check described in Sections 8.4–8.7, completely unchanged — the packet format and verification pipeline were designed to be provisioning-method-agnostic from the outset.

9. Core Algorithms

9.1 Time-to-Collision Prediction

``CollisionEngine::predictTTC()`` implements a constant-velocity closing-speed model between the local vehicle and one tracked peer. Both vehicles' speeds are converted from km/h to m/s and decomposed into East/North velocity components using compass heading (0° = North, 90° = East). The peer's position offset is converted from latitude/longitude degrees into metres using an equirectangular approximation — accurate at the short ranges (tens to low hundreds of metres) at which V2V collision warning is meaningful, and computationally far cheaper than a full great-circle calculation for a computation repeated for every tracked vehicle, ten times a second.

```
// risk/CollisionEngine.cpp - predictTTC() (excerpt)
float relVx = v0thX - vSelfX, relVy = v0thY - vSelfY;    // relative velocity, self's
frame
float dx = (other.lon - self.lon) * 111000.0f * cosf(self.lat * DEG2RAD);
float dy = (other.lat - self.lat) * 111000.0f;
float dist = max(sqrtf(dx*dx + dy*dy), 1.0f);              // avoid divide-by-zero at very
close range
```

```

float closingSpeed = (relVx * dx + relVy * dy) / dist;    // projection of rel. velocity
onto line of sight
if (closingSpeed <= 0.1f) { outTTC = 999.0f; return false; } // diverging or parallel: not
a threat

outTTC = dist / closingSpeed;
return outTTC < thresholds::TTC_CRITICAL;                // 1.5 s

```

The closing speed is the projection of the relative velocity vector onto the line-of-sight vector between the two vehicles, so a vehicle passing alongside at speed but on a diverging or parallel path correctly yields a very large TTC (the model clamps the sentinel value to 999.0 seconds) rather than a spuriously short one. A TTC below 1.5 seconds is treated as critical and drives the local alert level directly to Emergency; a TTC below 4.0 seconds (but at or above 1.5) is treated as a warning and contributes to a Danger-level alert. Because the calculation is re-run against every tracked vehicle on every 10 Hz network-task cycle, the reported minimum TTC across all peers is what the dashboard and alert logic ultimately act on.

9.2 Fused Accident Confidence Scoring

`AccidentDetector::update()` computes five boolean signals every IMU cycle from the current sample and derived deceleration:

Table 6. Individual accident-detection signals and their confidence contribution.

Signal	Trigger condition	Confidence weight
Impact	linear acceleration $\geq 18.0 \text{ m/s}^2$	+35
Severe impact	linear acceleration $\geq 26.0 \text{ m/s}^2$	+15 (additive to Impact)
Violent rotation	angular rate magnitude $\geq 160.0 \text{ }^\circ/\text{s}$	+20
Speed drop	deceleration $\geq 7.0 \text{ m/s}^2$	+20
Rollover orientation	$ \text{roll} \text{ or } \text{pitch} \geq 50.0^\circ$	+30
Poor recent behaviour	driving score < 50	+5

The raw sum is scaled by 0.75 whenever position status is not Available (i.e. the vehicle is in dead-reckoning mode), reflecting the reduced trust placed in a deceleration signal that is itself derived from an estimated rather than measured speed, and the result is clamped to the 0–100 range. Crucially, a high confidence score alone never triggers confirmation. A `multiSignal` condition additionally requires either (impact together with rotation or speed-drop) or (rollover together with rotation); a `fusedEvent` fires only when `multiSignal` is true, or when a plain impact has persisted for at least three consecutive abnormal samples (`REQUIRED\_ABNORMAL\_SAMPLES`). This is what allows the detector to reject a single hard pothole strike — which produces a strong, brief impact spike with no accompanying rotation, deceleration, or sustained abnormality — while still reacting quickly to a genuine, multi-signal collision or rollover event.

Once a `fusedEvent` first fires from the Monitoring state, the detector enters Confirming and starts a ten-second countdown (`CONFIRMATION\_COUNTDOWN\_MS`). While Confirming, every further `fusedEvent` refreshes an evidence timestamp; if no fresh evidence arrives within a two-second window (`EVIDENCE\_WINDOW\_MS`) and the vehicle is not currently in a rollover orientation, the state reverts to Monitoring — a genuine but brief event that does not continue to accumulate evidence is treated as a false alarm rather than being force-confirmed purely by the passage of time. Only if the countdown elapses with at least four confirming samples recorded (`MIN\_CONFIRMING\_SAMPLES`) and the peak confidence reached at least 60 does the state advance to Confirmed, at which point it latches — no further update can revert a Confirmed accident — and a one-shot confirmation flag is raised for the network task to consume.

9.3 GPS-Loss Dead Reckoning

When `PositionEstimator::update()` receives a sample with `gpsValid == false` but a previously valid fix exists, it integrates a fresh position estimate purely from inertial data:

```
// gps/PositionEstimator.cpp – dead-reckoning branch (excerpt)
state.headingDeg += data.gzDps * dt; // integrate yaw rate
float speedMps = clamp(state.speedKmh/3.6f + data.ax*dt, 0, 55) *
gps::DEAD_RECKONING_SPEED_DECAY; // 0.998
state.speedKmh = speedMps * 3.6f;
float distance = speedMps * dt;
state.lat += (cosf(heading_rad) * distance) / 111111.0f;
state.lon += (sinf(heading_rad) * distance) / (111111.0f * cosf(state.lat * DEG2RAD));
state.confidence -= (status == WEAK ? 1 : 2); // decays faster once fully
Lost
```

The 0.998 per-tick speed decay is a deliberate, conservative damping term: without it, any residual accelerometer bias not fully removed by boot-time calibration would integrate into an unbounded, ever-growing speed estimate over a long GPS outage; with it, the estimated speed exponentially relaxes toward zero in the absence of further acceleration, trading a small amount of dead-reckoning accuracy on a long straight-line outage for guaranteed boundedness. The decaying confidence value, rather than a binary valid flag, is what allows the accident detector to remain usefully active — at reduced confidence — throughout a GPS outage instead of either trusting an old fix indefinitely or disabling accident detection the moment GPS is lost.

9.4 Dual-Impulse Pothole Discrimination

`Pothole Detector::update()` maintains an exponentially smoothed baseline of vertical acceleration (`0.94 × baseline + 0.06 × sample`) and computes `impact = verticalLinearAccel - baseline` on every IMU cycle, but only while GPS is valid, the vehicle is moving at 8 km/h or more, and no cooldown is active. A real pothole produces a characteristic two-phase signature — the wheel drops (a strong impulse in one direction) and then rebounds against the far edge of the pothole or the suspension's own return travel (a

strong impulse in the opposite direction) within a short window — which is distinct from both routine road-texture vibration (too small in magnitude) and a long, gentle suspension sway over a speed bump (same sign, not a rapid reversal). The detector tracks the sign and peak magnitude of the first impulse that exceeds the 4.5 m/s^2 threshold within a 650 ms window; if a second impulse of the opposite sign arrives within that same window, the event is confirmed, its severity is computed from the combined peak magnitude of both impulses scaled against the threshold, and a seven-second cooldown begins so the same physical pothole is not re-reported on every subsequent axle or wheel passing over it.

9.5 Composite Driver Risk Score

The composite formula implemented in `DriverRiskEngine::update()` is reproduced here exactly as coded, since it is the single number most directly responsible for the vehicle's overall AlertLevel:

```
behaviorRisk = round((100 - drivingScore) * 0.55)
  + (unsafeLaneChangeActive ? 12 : 0)
  + (swervingActive         ? 10 : 0)
  + (aggressiveSteerActive  ?  8 : 0)

alcoholRisk = (alcoholLevel == CONFIRMED_HIGH) ? 45
              : (alcoholLevel == POSSIBLE)      ? 20 : 0

total = behaviorRisk + alcoholRisk + (harshBrakeThisCycle ? 8 : 0)
if (accidentStatus == CONFIRMED) total = 100
elif (accidentStatus == CONFIRMING) total += 18
total = clamp(total, 0, 100)

level = total >= 80 ? CRITICAL : total >= 55 ? DANGEROUS : total >= 25 ? MODERATE : SAFE
```

Because every term in this sum corresponds to a named, independently observable condition, an investigator or developer reviewing a black-box record can reconstruct exactly why a given risk score and alert level were reached at the moment of an event, without needing to reason about an opaque model — a property that matters for a system whose output can directly precede a tamper-evident evidence capture.

10. Data Persistence and Offline Datasets

10.1 Filesystem

All on-device persistence uses LittleFS, mounted at boot with `LittleFS.begin(true)` - the `true` argument requests an automatic format if the filesystem is not already valid, which allows a factory-fresh or corrupted flash to still boot into a usable, if empty, state rather than refusing to start. Three categories of data live on the filesystem: the seed datasets (`/hazards.csv`, `/places.csv`) uploaded once via `pio run -t uploaddfs`; the runtime-maintained binary stores derived from those seeds plus locally and remotely detected potholes (`/hazards.db`); and the rotating black-box evidence files (`/crash0.bin` through `/crash2.bin`).

10.2 Hazard Dataset Format

`/hazards.csv` accepts four comma-separated columns - type, latitude, longitude, severity - with lines beginning with `#` treated as comments and skipped. The supported hazard types are accident-prone zone, sharp curve, speed breaker, railway crossing, school zone, and pothole, mapped case-insensitively from several accepted text spellings by `hazardTypeFromText()`. On import, every valid row is inserted through the same `addHazard()` path used for runtime pothole detections, so a seeded record and a detected record are indistinguishable once stored, and duplicate suppression within a 12-metre radius applies equally to both

10.3 Facility Dataset Format

`/places.csv` accepts four columns - type, name, latitude, longitude - for the four supported facility types (hospital, police, fire, fuel), again parsed case-insensitively with several accepted synonyms per type (for example, both "fire" and "fire_station"). The project's design documentation is explicit that the repository intentionally ships without inventing plausible-looking hazard or facility coordinates for any specific region: both datasets are expected to be field-provisioned for the actual deployment area before the system is put into service, and an unprovisioned facility request correctly reports "no local record" rather than silently returning nothing or a stale placeholder.

10.4 Atomic Write Pattern

Both `HazardMap::saveToStorage()` and `DigitalBlackBox::writeCrashFile()` follow the same atomic-write discipline: write the new content to a `.tmp` path, verify every byte was written successfully, and only then remove the previous file at the real path and rename the temporary file into place. Because `LittleFS` rename is effectively atomic at the flash block level, a power loss at any point before the final rename leaves the previous, still-valid file untouched, and a power loss during the rename itself leaves either the old or the new file intact - there is no window in which a reader can observe a truncated or half-written file at the canonical path.

10.5 Integrity Verification

Both persisted stores are protected against silent corruption or tampering, using two different but structurally similar mechanisms appropriate to their differing size and use: the hazard store uses a CRC-32 checksum over its fixed-size record array, checked against magic and version fields on every load, which is fast and sufficient for detecting accidental corruption of a store that is rewritten frequently; the black-box evidence files use a full SHA-256 hash over the header and every sample, checked at every boot, which markedly more expensive but appropriate for a store whose entire purpose is to serve as tamper-evident evidence rather than merely internally-consistent cached data.

11. Testing, Validation, and Debugging

11.1 Review Methodology

The codebase underwent a structured, full-repository review pass covering every module described in Chapter 7, focused specifically on three defect classes of outsized consequence in a safety-relevant

embedded system: correctness defects that would silently prevent a subsystem from functioning at all, memory-safety defects reachable from untrusted input (in this system, that means any ESP-NOW packet from any device in radio range, since the platform has no physical pairing step), and availability defects that could cause an unplanned reset or hang. The review was paired, where practical, with static reasoning about the exact numeric thresholds and state machines documented in Chapter 9, since a plausible-looking but subtly wrong threshold is a more insidious defect in this kind of system than an outright crash.

11.2 Defects Identified and Corrected

Eight defects of material consequence were identified and corrected during this review. They are summarised in Table 5, ordered by discovery, together with the class of defect and its practical impact had it shipped uncorrected.

Table 7. Defects identified during code review, their class, and their pre-fix impact.

#	Defect	Class	Pre-fix impact
1	Self-referential checksum in the outbound packet path	Correctness	Every transmitted V2V packet failed the receiver's own integrity check, making inter-vehicle communication completely non-functional despite the radio link itself working correctly.
2	Unvalidated, attacker-controlled `encLen` field on receive	Memory safety	A crafted ESP-NOW packet from any device within radio range could declare a ciphertext length exceeding the true payload size, overflowing a fixed 38-byte structure during decryption — a remotely triggerable memory-corruption vulnerability requiring no prior authentication.
3	Single, globally shared replay-protection window	Correctness / availability	Because the sliding replay window was tracked once for the whole fleet rather than per sender, the first vehicle heard from in a session advanced the shared window far enough that every other vehicle's independently numbered, legitimately lower sequence numbers were then rejected as replays — permanently locking out all but one vehicle per session.
4	Watchdog task registration fault	Availability	One or more FreeRTOS tasks were not correctly registered with the hardware watchdog, causing a panic-reboot roughly five seconds after every boot once the watchdog timeout elapsed on an unregistered or mis-registered task.
5	Missing MAC-address-to-identity binding on receive	Security hardening	Without binding the claimed vehicle ID inside a packet to the sender's actual link-layer MAC address, a device could potentially transmit packets claiming another vehicle's derived identity.
6	Incomplete authenticated-encryption associated-data binding	Security hardening	Strengthened so that the GCM authentication additionally covers the full unencrypted header, preventing header fields from being altered

#	Defect	Class	Pre-fix impact
			independently of the encrypted payload without detection.
7	Sequence-counter persistence gap	Availability / security	The monotonic per-vehicle sequence counter, on which nonce uniqueness depends, was not reliably reserved and persisted across reboots, risking nonce reuse — which would break AES-GCM's confidentiality and integrity guarantees — after an unclean shutdown.
8	Ordering of authentication relative to encryption	Security hardening	The send and receive paths were corrected to a consistent encrypt-then-authenticate / verify-then-decrypt ordering (Sections 8.4–8.5), so that a forged or corrupted packet is rejected by the cheaper HMAC check before any AES-GCM decryption is attempted.

Defect 1 alone is worth dwelling on: because it manifested as every packet silently failing receiver-side validation rather than as a crash or an obvious error, the system would have appeared to build, flash, and run correctly — the dashboard, sensors, and local alerting all continue to function — while providing none of its core inter-vehicle safety value. This is precisely the kind of defect that unit testing of individual functions is prone to miss and that requires an end-to-end, two-device validation step (Section 11.4, item 9) to surface.

11.3 Resource Footprint

The verified build for the `mhetesp32devkit` target consumes 74,444 bytes of static DRAM, 22.7% of the 320 KB conventionally available to the application in the default partition layout, and 905,301 bytes of flash, 69.1% of the 1,310,720-byte default application partition, leaving meaningful headroom for the future extensions discussed in Chapter 14 without requiring a partition-table change. The LittleFS data partition is a fixed 1,441,792-byte image, sized to comfortably hold the hazard and facility datasets, the runtime hazard-store binary, and all three rotating black-box evidence slots simultaneously.

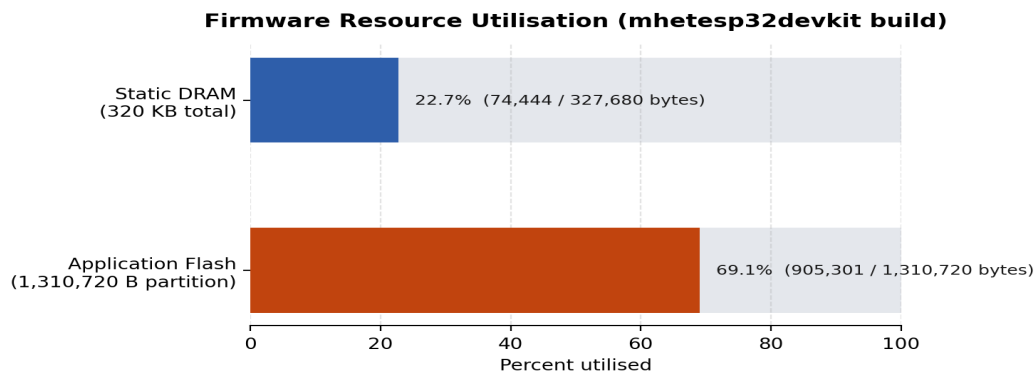


Figure 3. Static DRAM and flash utilisation of the verified `mhetesp32devkit` build.

11.4 Validation Checklist

The following field-validation sequence is specified for bringing a new physical unit into service and for verifying firmware changes before fleet-wide deployment:

- Build with ``pio run``; confirm the target is ``mhetesp32devkit`` and the build completes without warnings escalated to errors.
- Run ``pio run -t uploadfs`` after populating real, field-specific hazard and facility datasets.
- Confirm the SIM28 receiver is producing NMEA sentences and that the TFT dashboard shows a valid coordinate.
- Keep the vehicle stationary through the IMU and MQ-3 calibration windows at boot, so bias and baseline estimates are not corrupted by motion or transient gas exposure.
- Exercise lane, braking, pothole, alcohol, and accident thresholds in a closed area before any road use, confirming the physical mounting orientation matches the X-forward/Y-left/Z-up assumption documented in Section 5.4.
- Confirm a single IMU impact alone does not trigger emergency handling, and that a sustained multi-signal event correctly reaches the ten-second confirmation countdown; test both a mid-countdown cancellation and a full confirmation on a test fixture.
- Disconnect the GPS antenna after acquiring a valid fix and confirm the dashboard correctly reports Weak then Lost status, that estimated navigation and decaying confidence behave as documented in Section 9.3, and that the system correctly resynchronises to a live fix on reconnection.
- Confirm MQ-3 warm-up behaviour, a stable Possible-risk classification, five consecutive stable high readings reaching Confirmed-High, and that the authenticated impaired-driver warning is correctly received by a second fleet unit.
- Flash two units with the same fleet root key and verify end-to-end V2V telemetry, lane and pothole alerts, replay rejection under a deliberately re-sent packet, and emergency broadcast — this two-device step is specifically what would have caught Defect 1 in Table 5 had it been run before the review.
- Simulate an accident event only on a test fixture, then inspect the resulting ``crash0.bin`` through ``crash2.bin`` files; deliberately modify one file on disk and reboot to confirm the dashboard correctly reports an integrity-failed state for that record.

12. Results and Discussion

12.1 Functional Coverage Against Requirements

All functional requirements listed in Section 3.1 are implemented by a directly-traceable module detailed in Chapter 7: behaviour scoring by Driver Behaviour Analyzer, impairment classification by Alcohol Risk Engine, accident confirmation by Accident Detector, dead reckoning by Position Estimator, pothole discrimination by Pothole Detector, authenticated peer messaging by V2VManager and Security

Manager together, collision prediction by Collision Engine, hazard mapping by Hazard Map, offline routing by Offline Navigation, health aggregation by Vehicle Health Monitor, tamper-evident logging by Digital BlackBox, and local presentation by Dashboard and Alert Manager. No functional requirement in Section 3.1 relies on any connectivity beyond that provided by the ESP-NOW link between already-present, in-range vehicles, thus satisfying the core platform design goal.

12.2 Resource and Timing Margins

With 22.7% DRAM and 69.1% flash utilisation at the current feature set (Section 11.3), the firmware still has substantial headroom for the roadside-receiver, OBD-II and OTA extensions discussed in Chapter 14 without the need for immediate partitioning rearrangement beyond the standard setup. From a timing perspective, the four-task, dual-core partition (Chapter 6) successfully separates the highest frequency work (IMU sampling at 50 Hz) from the highest-latency work (LittleFS writes and ESP-NOW transmission), and all tasks meet the five-second watchdog on each pass with margins at least a decade more aggressive than the timeout, making a single task stall unlikely to result in a watchdog reset.

12.3 Behaviour of the Fusion Logic Under the Validation Checklist

The multi-signal accident-confirmation design (Section 9.2) is directly validated, in line with checklist item 6, against the most likely cause of false positives in a real-world, road-based IMU crash detector: an isolated pothole or kerb-strike causing a sharp spike but no genuine deceleration or rotation. Since fused Event either requires the presence of a signal pattern consistent with an actual accident or requires three consecutive off-spec samples of plain impact, a momentary, non-persistent impact that is unaccompanied by rotation or speed loss will not pass into the Confirming state let alone the Confirmed state (which would be reached after a ten-second countdown); conversely, a collision - which characteristically features impact, rotation, and deceleration simultaneously - satisfies the multi Signal criterion in the very first sample where one is met and starts the countdown timer

12.4 Security Posture After Remediation

Following the corrections detailed in Table 5, the send and receive paths employ a conventional and auditable authenticated-encryption scheme: on the send side, it's encrypt-then-MAC; on the receive side, the logic is structural-validation > MAC-to-identity-binding > HMAC-verification > AES-GCM-decryption > replay-window-check (Sections 8.4-8.5). All checks are hard-fail, with no partial trust allowed, and the flow optimises this by only performing the computationally expensive AES-GCM-decryption after simpler structural and authentication checks have passed. This places a cap on the number of computationally intensive computations an attacker can force a receiver to perform by simply bombarding it with fabricated or poorly-formed messages, a lightweight and ESP-NOW-suited attack vector against denial-of-service style flooding which should be considered alongside the limitation on rate-limiting discussed in Chapter 13.

13. Advantages

- Zero connectivity dependency for every safety-relevant function, making the platform effective in exactly the highway, rural, hill, and tunnel environments where cloud-dependent systems degrade.
- Low per-unit cost relative to dedicated V2X radio hardware, using only an ESP32 module and a small set of commodity sensors already produced at very large scale for other markets.
- Multi-signal accident confirmation that is explicitly designed and validated to reject the most common single-signal false-positive case (an isolated pothole or kerb impact).
- A tamper-evident, cryptographically hashed evidence trail that is independently verifiable at every boot without needing to trust the device that produced it, using only the file itself.
- A symmetric key architecture appropriate to hardware with no interactive pairing UI, with an explicit, wire-format-compatible upgrade path to authenticated ECDH commissioning.
- A modular, snapshot-isolated software architecture (Section 4.4) in which every subsystem can be reasoned about, and in principle tested, independently of the others.
- Field-maintainable offline datasets (CSV) that do not require a firmware rebuild to update hazard or facility information for a new deployment area.

14. Future Scope

14.1 Roadside Receiver Units

A fixed, mains- or solar-powered ESP-NOW receiver deployed at known high-risk points (accident-prone bends, unmanned railway crossings, school zones) could passively log passing-vehicle telemetry and, more importantly, could itself broadcast static hazard information to any vehicle unit passing within range, extending the effective warning horizon beyond what two moving vehicles happening to be in simultaneous radio range can provide alone.

14.2 Emergency-Service Integration

The Emergency-priority broadcast already carries position, crash type, and confidence; a gateway device bridging ESP-NOW to a cellular or LoRaWAN uplink at a fixed location, or carried in a dispatch vehicle, could relay a confirmed-accident broadcast to an emergency dispatch system substantially faster than a bystander placing a manual call, without requiring every vehicle unit itself to carry a cellular modem.

14.3 Fleet Management and Analytics

The vehicle-health, driver-risk, and black-box evidence data already computed on-device is structured and complete enough to support a periodic (non-real-time) bulk offload - for instance, over a wired connection during scheduled maintenance - into a fleet analytics system, without requiring the real-time connectivity the core safety functions deliberately avoid depending on.

14.4 OBD-II and CAN Bus Integration

Direct access to true wheel speed, throttle position, brake-pedal state, and fault codes from the vehicle's own OBD-II or CAN bus would materially improve the confidence and precision of several thresholds currently derived purely from GPS-reported speed and IMU-estimated deceleration, particularly the speed-drop signal used in accident confirmation (Section 9.2) and the harsh-braking detector used in driver-behaviour scoring (Section 7.2).

14.5 Over-the-Air Firmware Updates

A signed OTA update mechanism, consistent with the authenticated-encryption discipline already applied to V2V telemetry, would allow threshold tuning, bug fixes, and new hazard-type support to be deployed to an existing fleet without a physical USB connection to every unit - valuable specifically because the platform's target deployment geography is the same low-connectivity environment that makes remote field service logistically difficult.

14.6 Camera-Assisted Driver Monitoring

A low-cost camera module addressing driver drowsiness and distraction (eye closure, head pose, phone use) would complement the platform's existing inertial-and-gas-sensor-only behavioural signal with a class of impairment - fatigue and distraction - that neither the IMU nor the MQ-3 sensor can observe.

14.7 Smart Traffic Infrastructure Integration

In regions where signalised intersections or variable-message signs are instrumented with their own low-power radio, an ESP-NOW-to-infrastructure bridge analogous to the roadside receiver concept in Section 14.1 could allow Drew's Drive units to receive infrastructure-originated warnings (an approaching emergency vehicle, a signal fault, a temporary closure) in addition to peer-originated ones, moving the platform incrementally closer to a full V2X posture while preserving its offline-first core.

15. Conclusion

Drew's Drive demonstrates that a materially useful subset of connected-vehicle safety functionality — real-time driver-behaviour scoring, non-invasive impairment flagging, multi-signal fused accident confirmation, authenticated peer-to-peer collision warning, offline hazard and facility guidance, and tamper-evident evidence recording — can be delivered entirely on a single low-cost microcontroller with no dependency on internet, cellular, or roadside infrastructure connectivity. The architecture achieves this through a disciplined, snapshot-isolated software design across nineteen cooperating modules, a symmetric but sender-derivable security architecture appropriate to hardware with no interactive pairing channel, and fusion algorithms whose thresholds and state machines are explicit and independently inspectable rather than opaque.

The structured review documented in Chapter 11 — which located and corrected eight defects spanning a communication-breaking checksum bug, a remotely triggerable buffer overflow, a fleet-locking replay-protection flaw, and a watchdog registration fault, among others — underscores that a safety-relevant embedded system of this scope benefits materially from a review process that specifically probes correctness under adversarial and multi-device conditions, not only single-device functional testing. With those defects corrected, the platform's resource footprint (22.7% DRAM, 69.1% flash) leaves comfortable headroom for the roadside-infrastructure, fleet-analytics, and OBD-II extensions identified in Chapter 14, positioning Drew's Drive as a practical, retrofit-affordable safety layer for the connectivity-poor road environments where the need for exactly this kind of system is greatest.

References

- [1] Espressif Systems, ESP32 Series Datasheet and ESP-IDF Programming Guide — ESP-NOW protocol, Wi-Fi radio, and hardware AES/SHA acceleration.
- [2] InvenSense (TDK), MPU-6000 and MPU-6050 Product Specification and Register Map — six-axis MotionTracking device used for the platform's inertial sensing.
- [3] SIMCom Wireless Solutions, SIM28 GPS Module Hardware Design and AT Command documentation — NMEA-0183 sentence output used for positioning.
- [4] Winsen Electronics, MQ-3 Semiconductor Alcohol Gas Sensor datasheet and characteristic curves — sensitivity-to-resistance-ratio relationship used for the ambient alcohol-risk estimate.
- [5] ARM/Trusted Firmware Project (mbedTLS), mbedTLS Cryptographic Library documentation — AES-GCM authenticated encryption and HMAC-SHA-256 / SHA-256 primitives used throughout the security architecture.
- [6] NIST Special Publication 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC — authenticated-encryption construction referenced in Chapter
- [7] IEEE 802.11p / DSRC and 3GPP Cellular-V2X standardisation documentation — surveyed as related connected-vehicle radio technologies in Chapter 2.
- [8] FreeRTOS Kernel documentation (Amazon Web Services) — task scheduling, mutex (semaphore), and queue primitives used across the firmware's four-task architecture.
- [9] LittleFS Project documentation — power-loss-resilient embedded filesystem used for on-device hazard, facility, and evidence persistence.
- [10] Project source repository: Drew's Drive firmware, README, and inline module documentation (github.com/ajaiandrews2006/drewsdrive) — the primary technical source for this report.

Appendix — Selected Source Listings

The listings below are reproduced from the current firmware to accompany the algorithmic discussion in Chapters 8 and 9, condensed to their operative logic.

B.1 Vehicle Identity and Key Derivation (security/SecurityManager.cpp)

```
bool hmacWithMac(const char* context, const uint8_t mac[6], uint8_t out[32]) {
    uint8_t material[40]{};
    size_t contextLength = strlen(context);
    memcpy(material, context, contextLength);
    memcpy(material + contextLength, mac, 6);
    return hmacSha256(FLEET_ROOT_KEY, sizeof(FLEET_ROOT_KEY), material, contextLength + 6,
out);
}

void SecurityManager::deriveLocalIdentity() {
    uint8_t mac[6]; WiFi.macAddress(mac);
    hmacSha256(FLEET_ROOT_KEY, sizeof(FLEET_ROOT_KEY),
        (const uint8_t*)NETWORK_CONTEXT, strlen(NETWORK_CONTEXT),
networkAesKey_digest);
    memcpy(networkAesKey, networkAesKey_digest, 16);           // fleet-wide AES-128 key
    hmacWithMac(AUTH_CONTEXT, mac, vehicleAuthKey);           // per-vehicle auth key
    hmacWithMac(ID_CONTEXT, mac, vehicleIdDigest);
    memcpy(vehicleId, vehicleIdDigest, 16);                   // per-vehicle identity
}
}
```

B.2 Replay-Window Check (security/SecurityManager.cpp)

```
bool SecurityManager::isReplay(const uint8_t senderId[16], uint32_t sequence) {
    if (sequence == 0) return true;
    ReplayState& state = findOrAllocate(senderId); // per-sender table, LRU eviction if
full
    if (sequence <= state.windowBase) return true; // at or behind the window: reject

    uint32_t distance = sequence - state.windowBase;
    if (distance > 256) { // far ahead: slide the window
forward
        state.windowBase = sequence - 256;
        memset(state.windowBits, 0, sizeof(state.windowBits));
    }
    uint32_t index = sequence - state.windowBase - 1;
    uint64_t mask = 1ULL << (index % 64);
    uint8_t word = index / 64;
    if (state.windowBits[word] & mask) return true; // already seen: reject
    state.windowBits[word] |= mask; // mark seen, accept
    return false;
}
}
```

B.3 Accident Fusion Decision (core/AccidentDetector.cpp)

```
bool multiSignal = (impact && (violentRotation || speedDrop)) ||
    (rolloverOrientation && violentRotation);
bool fusedEvent = multiSignal ||
    (impact && abnormalSamples >= accident::REQUIRED_ABNORMAL_SAMPLES);
```

```

if (fusedEvent) {
    lastEvidenceMs = now;
    if (state.status == MONITORING) {
        confirmationStartedMs = now;
        state.status = CONFIRMING;
    }
}
if (state.status == CONFIRMING) {
    if (now - lastEvidenceMs > accident::EVIDENCE_WINDOW_MS && !rolloverOrientation) {
        state = { MONITORING, CrashType::NONE, 0, 0, imuOnly }; // false-alarm reset
    } else if (now - confirmationStartedMs >= accident::CONFIRMATION_COUNTDOWN_MS &&
        confirmingSamples >= accident::MIN_CONFIRMING_SAMPLES &&
        state.confidence >= 60) {
        state.status = CONFIRMED; // latches; no further
reversion
    }
}
}

```

B.4 Pothole Rebound Confirmation (hazard/PotholeDetector.cpp)

```

float impact = data.verticalLinearAccel - verticalBaseline; // baseline: 0.94/0.06
exponential filter
float magnitude = fabsf(impact);
if (magnitude < pothole::IMPACT_THRESHOLD_MPS2) { /* below threshold: track window expiry
only */ }

int8_t sign = impact >= 0.0f ? 1 : -1;
if (impulseSign == 0 || now - impulseStartedMs > pothole::IMPULSE_WINDOW_MS) {
    impulseSign = sign; impulsePeak = magnitude; impulseStartedMs = now; // start
tracking impulse 1
} else if (sign == impulseSign) {
    impulsePeak = max(impulsePeak, magnitude); // same-
direction: update peak
} else {
    float severity = clamp((impulsePeak + magnitude) /
        (2 * pothole::IMPACT_THRESHOLD_MPS2) * 100.0f, 1, 100);
    pending = { true, data.lat, data.lon, severity, data.epochSec }; // opposite-
sign rebound: confirm
    cooldownUntilMs = now + pothole::COOLDOWN_MS;
}
}

```

B.5 Snapshot Accessor Pattern (representative of every stateful subsystem)

```

AccidentSnapshot AccidentDetector::snapshot() {

```

```
AccidentSnapshot copy;
xSemaphoreTake(mux, portMAX_DELAY);
copy = state;                // copy the whole struct under lock
xSemaphoreGive(mux);
return copy;                 // caller processes the copy lock-free
}
```

This exact pattern — take mutex, copy struct, give mutex, return by value — recurs verbatim across `PositionEstimator`, `AlcoholRiskEngine`, `DriverBehaviorAnalyzer`, `DriverRiskEngine`, `HazardMap`, `OfflineNavigation`, `VehicleHealthMonitor`, and `SensorHub`, and is the single architectural idiom most responsible for the codebase's freedom from lock-ordering deadlocks, discussed in Section 4.4.1.

Drew's Drive: Offline AI-Based Vehicle-to-Vehicle Collision Avoidance and Driver Safety System

TRL – TECHNOLOGY READINESS LEVEL

TRL 4: Component/breadboard validation in a laboratory environment.

AT TRL 4, INDIVIDUAL COMPONENTS HAVE MOVED BEYOND PAPER CONCEPT AND SIMULATION INTO VALIDATION UNDER CONTROLLED LAB CONDITIONS, BUT THE FULL SYSTEM HAS NOT YET BEEN INTEGRATED AND PROVEN IN A RELEVANT (REAL-WORLD DRIVING) ENVIRONMENT. DREW'S DRIVE SITS HERE: EACH FUNCTIONAL SUBSYSTEM EXISTS AS WORKING, TESTABLE FIRMWARE ON THE TARGET ESP32-WROOM-32 HARDWARE RATHER THAN A DESIGN ON PAPER.

SPECIFICALLY: THE SENSOR FUSION PIPELINE (GPS, IMU, MQ-3 ALCOHOL SENSOR) IS IMPLEMENTED AND CALIBRATABLE AGAINST REAL READINGS; ACCIDENT DETECTION COMBINES IMPACT, ANGULAR RATE, SPEED DROP, AND ORIENTATION THRESHOLDS WITH A WORKING 10-SECOND CONFIRMATION STATE MACHINE; THE V2V STACK RUNS REAL AES-128-GCM ENCRYPTION, HMAC-SHA-256 AUTHENTICATION, AND ANTI-REPLAY CHECKS OVER ACTUAL ESP-NOW RADIO HARDWARE, NOT A SIMULATED LINK; AND THE DIGITAL BLACK BOX WRITES AND CRYPTOGRAPHICALLY VERIFIES REAL FILES ON LITTLEFS STORAGE. A STRUCTURED CODE REVIEW ALREADY CAUGHT AND FIXED HARDWARE-RELEVANT DEFECTS (A PACKET VALIDATION BUG, A REMOTE BUFFER-OVERFLOW VECTOR, A WATCHDOG REGISTRATION FAULT) - ISSUES ONLY SURFACED ONCE CODE RUNS AGAINST REAL CONSTRAINTS, NOT PURELY DESIGNED.

DREW'S DRIVE TESTING MODULE

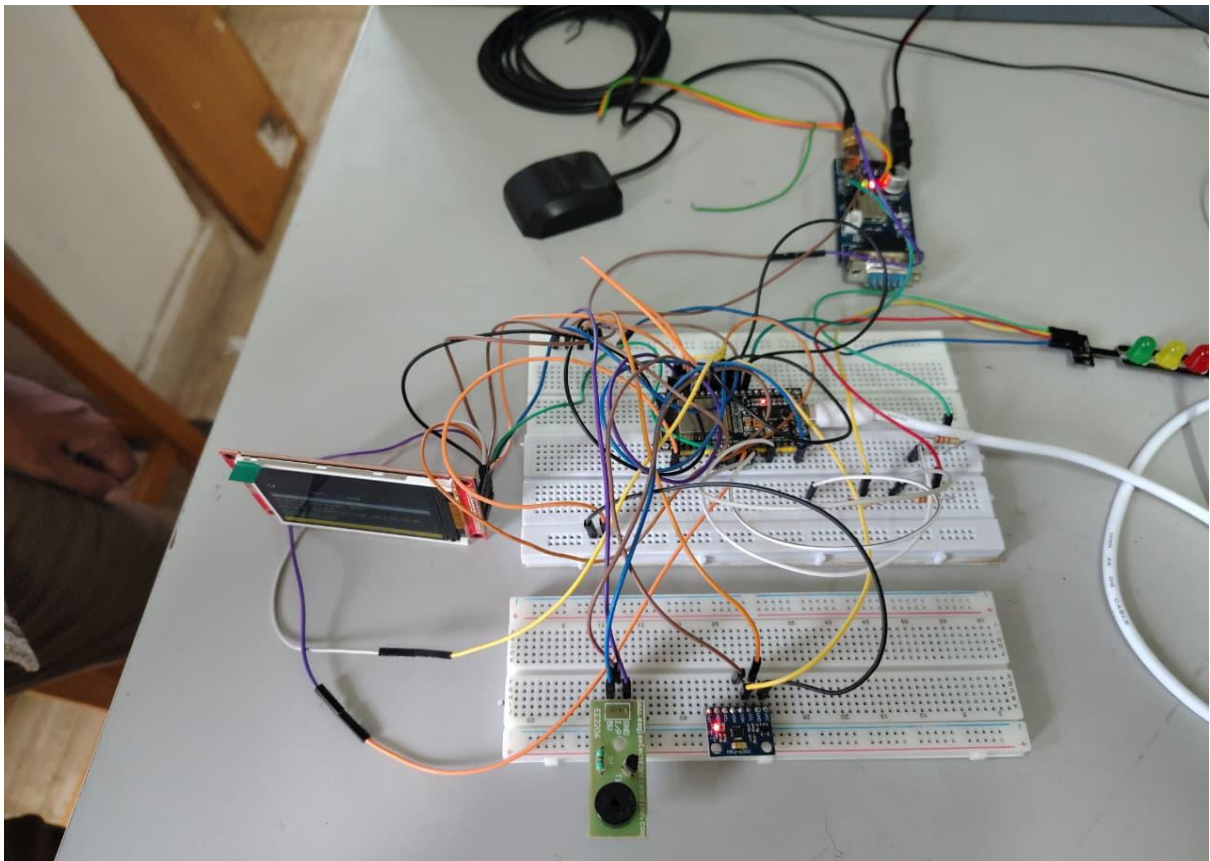


Fig 1:side view of testing module

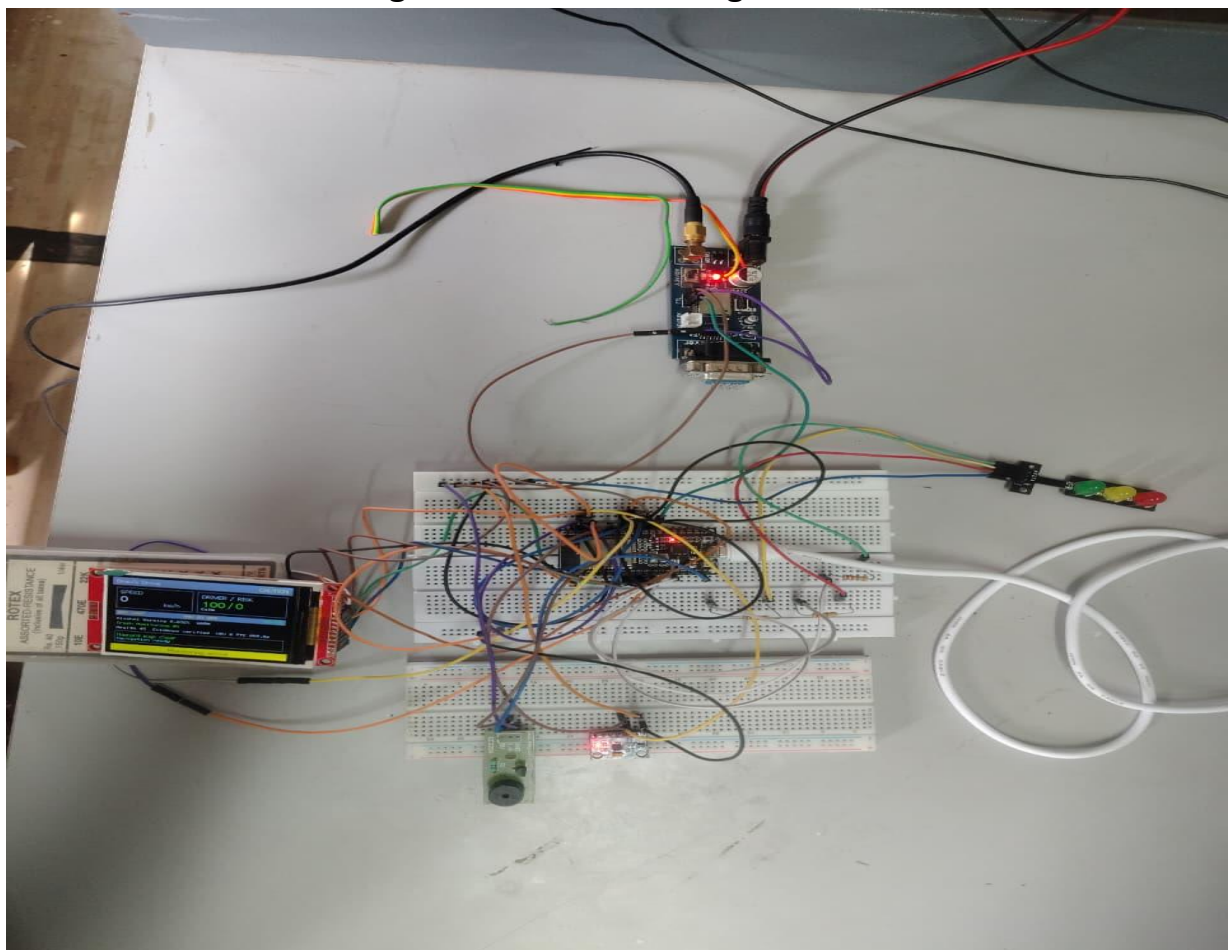


Fig 2:Top view of testing module



Fig: 3 Testing results

Drew's Drive: Offline AI-Based Vehicle-to-Vehicle Collision Avoidance and Driver Safety System

Architecture and Operational Workflow

S.NO	TITLE	PAGE
BLOCK DIAGRAMS		
1.	Overall Hardware Architecture of Drew's Drive	3
2.	Firmware Architecture of Drew's Drive	4
3.	Safety and Decision-Making Architecture	5
4.	Vehicle-to-Vehicle (V2V) Communication Architecture	6
5.	FreeRTOS Task Architecture	7
FLOW CHARTS		
6.	Firmware Execution Workflow	8
7.	Driver Safety Decision Workflow	9
8.	Emergency Response & Cooperative Warning Workflow	10

Overall Hardware Architecture of Drew's Drive

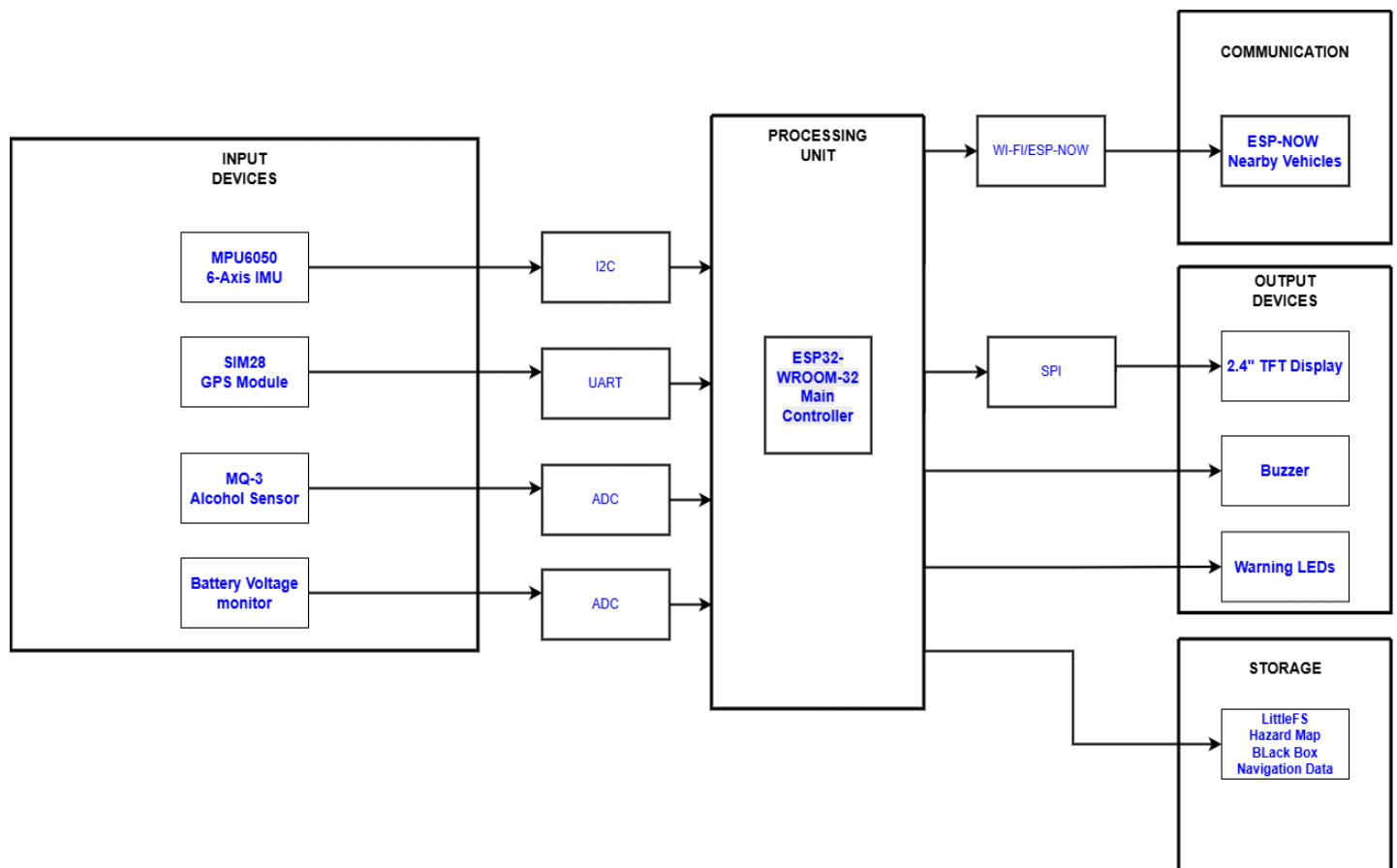


Figure 4.1 illustrates the overall hardware architecture of the Drew's Drive system

Firmware Architecture of Drew's Drive

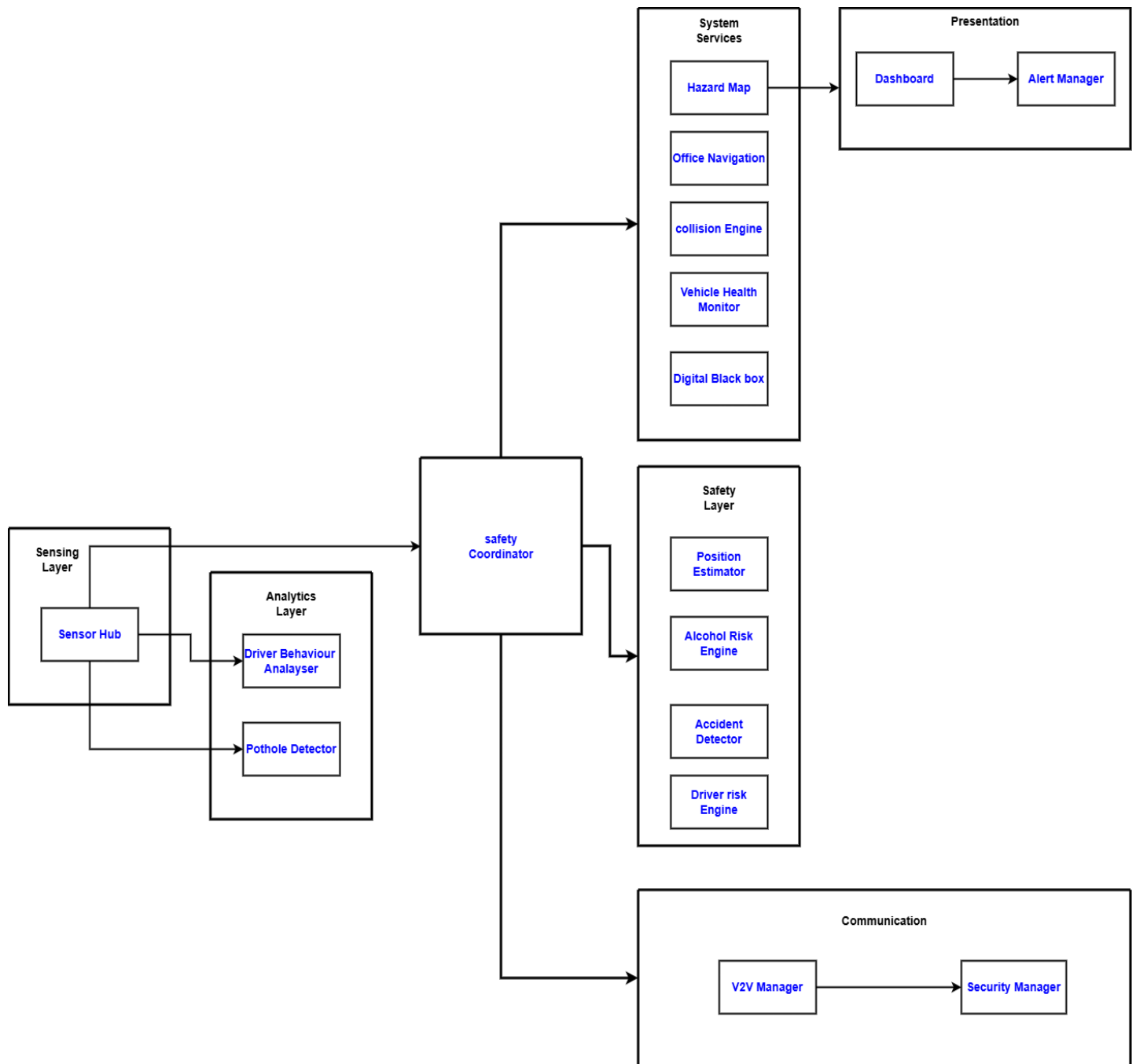


Figure 4.2 illustrates the major software modules and their interactions, demonstrating how sensor information is processed and coordinated

Safety and Decision-Making Architecture

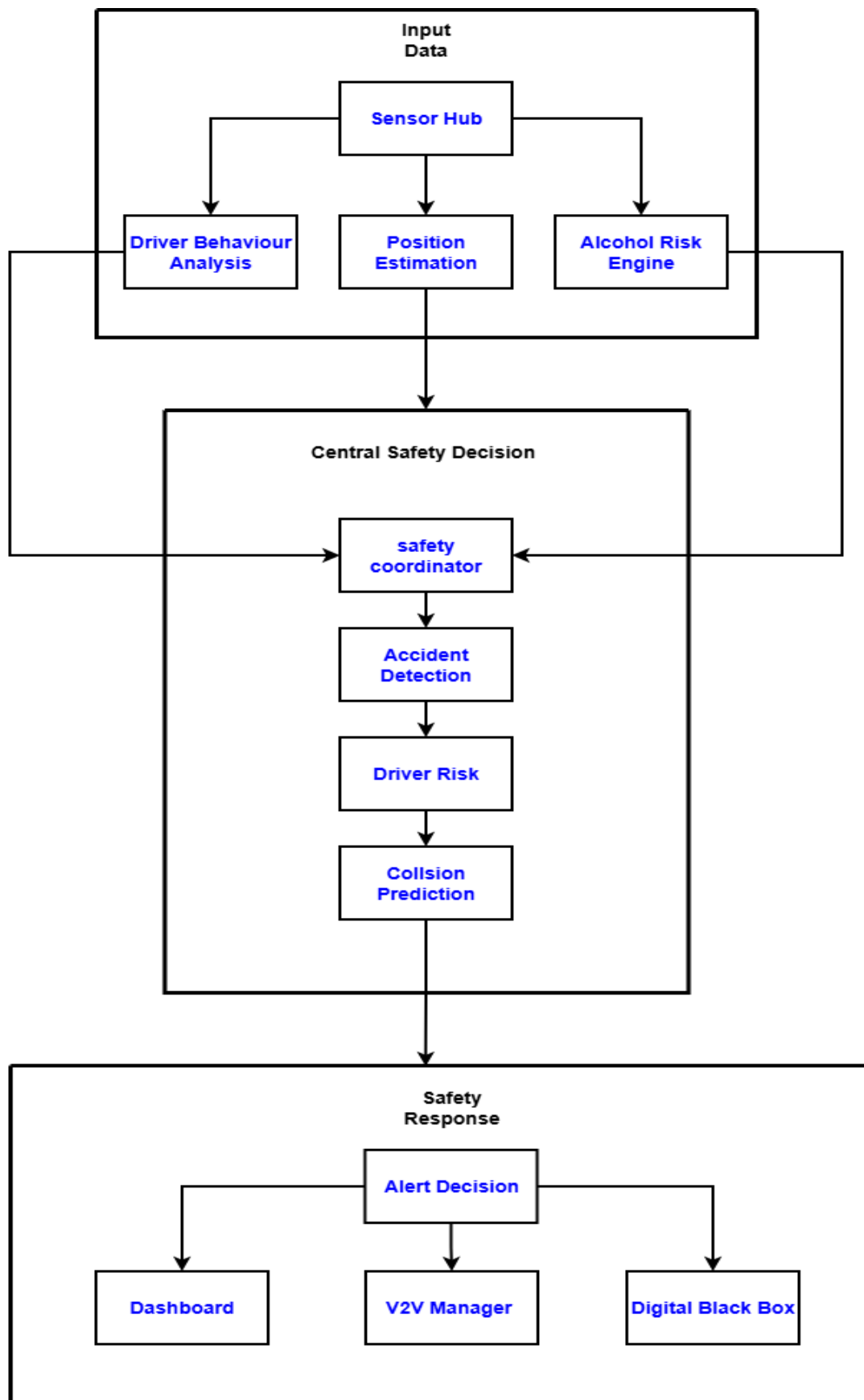


Figure 4.3 illustrates the safety decision-making architecture of Drew's Drive

Vehicle-to-Vehicle (V2V) Communication Architecture

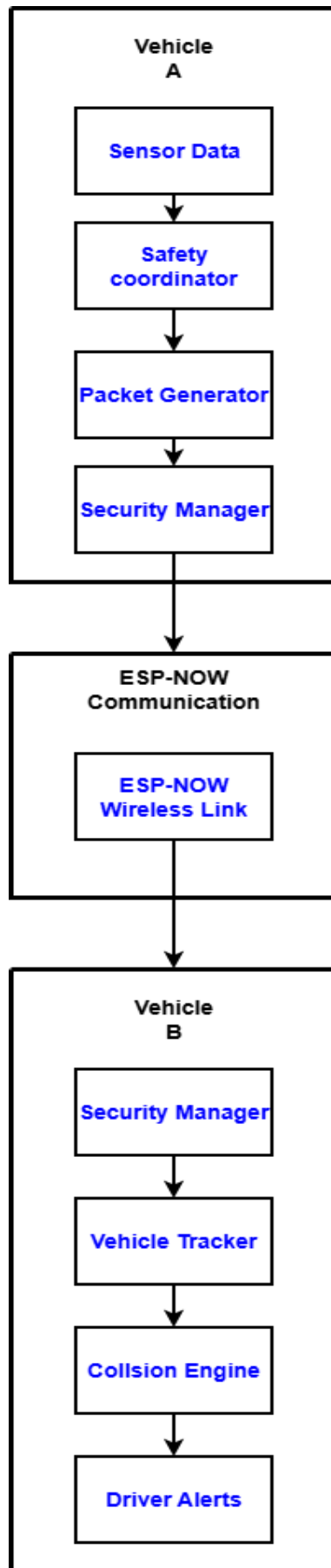


Figure 4.4 It illustrates the transmission of safety information between nearby vehicles using ESP-NOW communication, including packet generation, security processing, vehicle tracking, collision prediction, and driver notification.

FreeRTOS Task Architecture

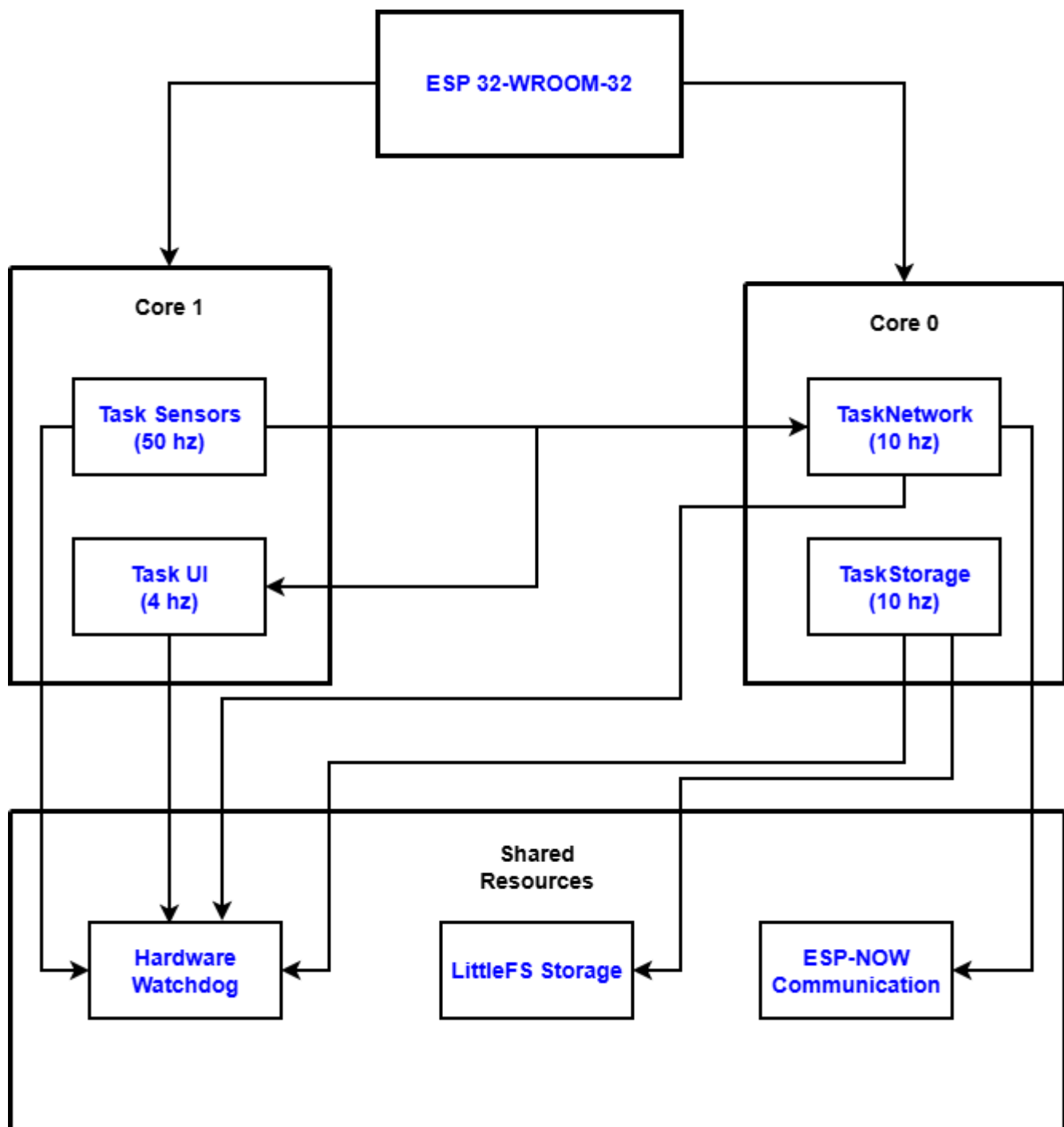


Figure 4.5 illustrates the real-time task architecture implemented on the ESP32 using FreeRTOS

Firmware Execution Workflow

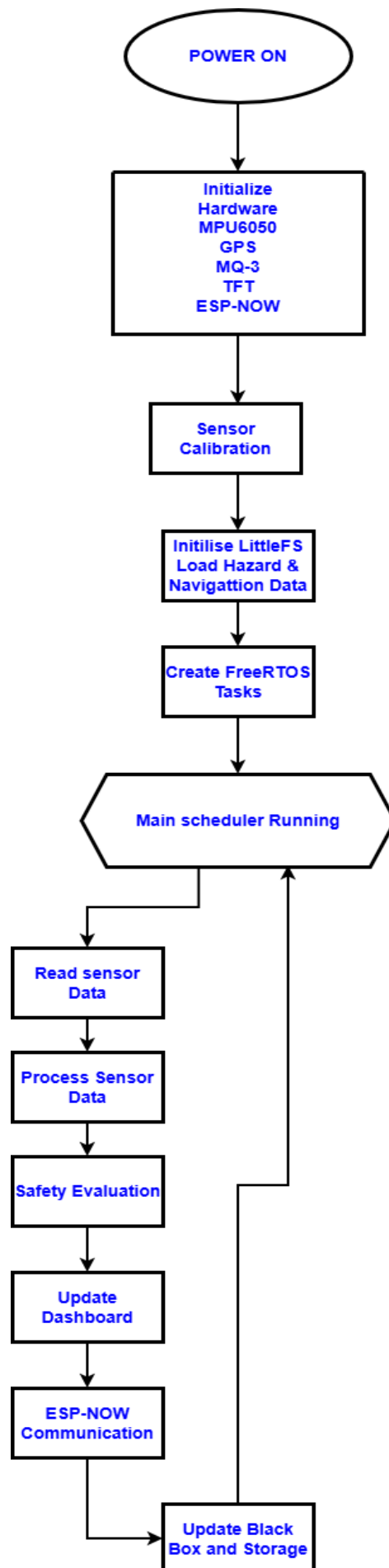


Figure 4.6 illustrates the overall firmware execution workflow of Drew's Drive

Driver Safety Decision Workflow

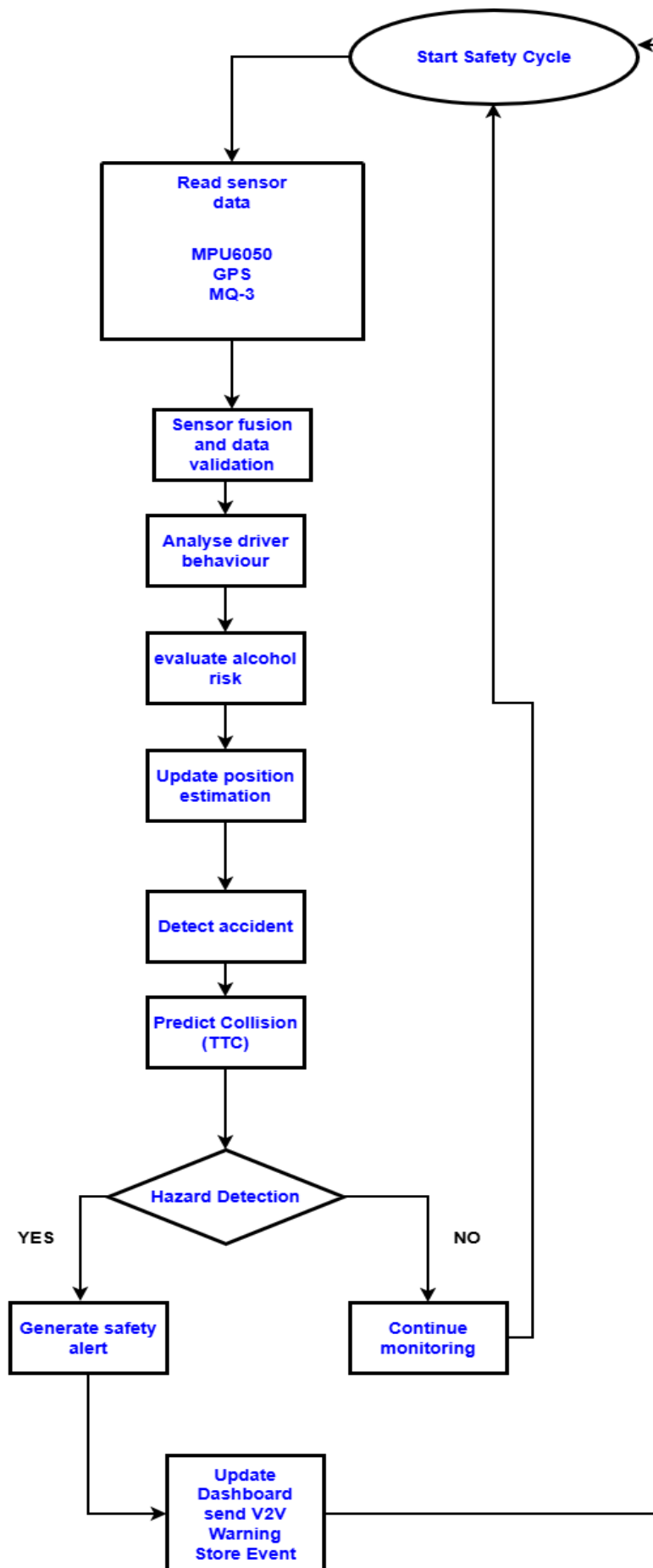


Figure 4.7 presents the driver safety decision workflow implemented in Drew's Drive

Emergency Response and Cooperative Warning Workflow

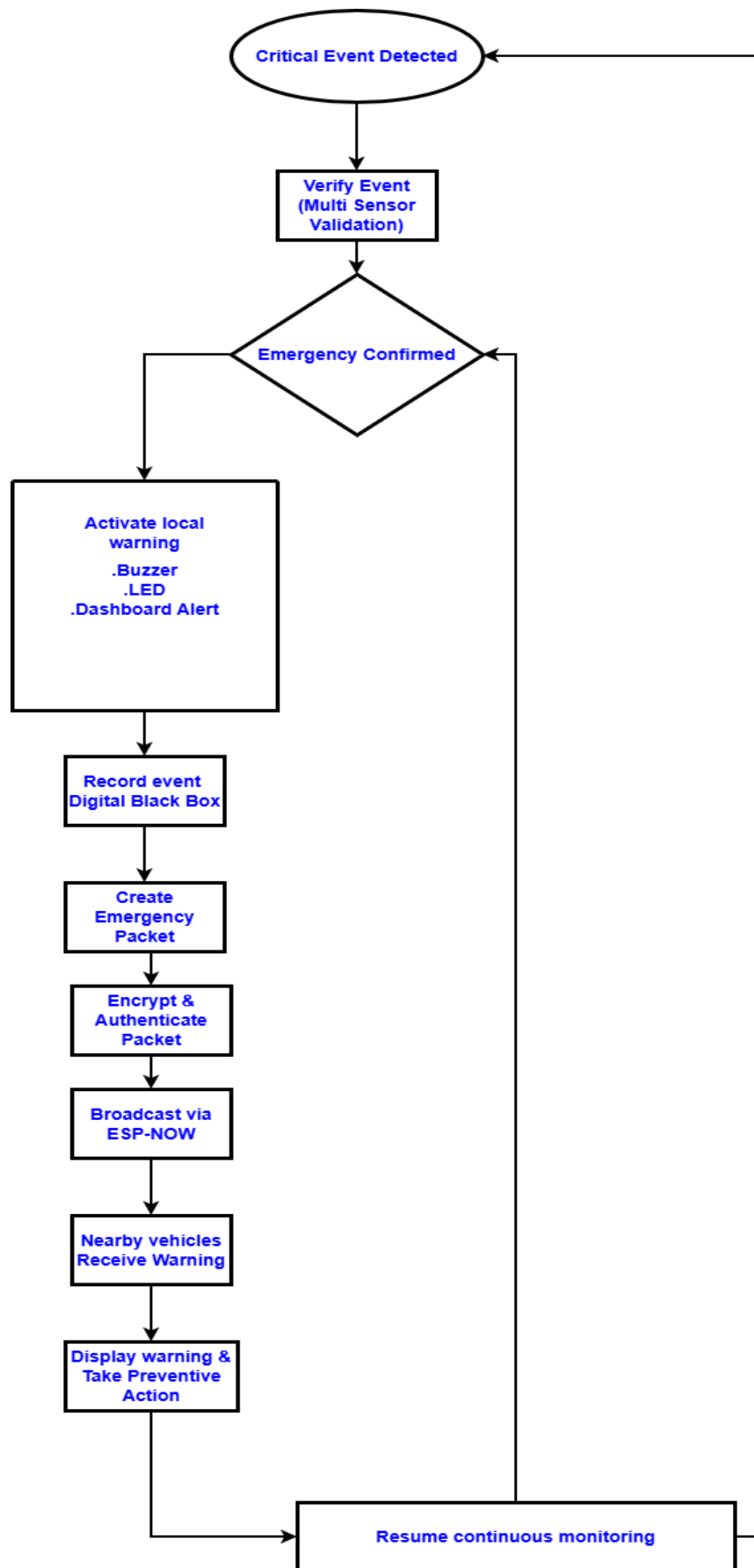


Figure 4.8 illustrates the emergency response workflow of Drew's Drive.

